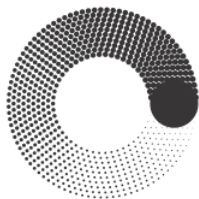


МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ



**Факультет Информационных технологий
кафедра Информатики и информационных технологий**

**направление подготовки 09.03.02 «Информационные системы и
технологии»**

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

БАКАЛАВРСКАЯ РАБОТА

Тема: Разработка сайта для настольно-ролевой игры

Исполнитель Гайдарь М.Д.
(Фамилия И.О.)

(Подпись)

Руководитель Кандрашина М.А.
(Фамилия И.О., степень, звание)

(Подпись)

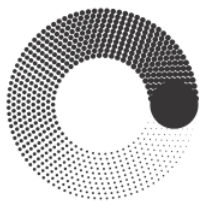
«ДОПУЩЕНО К ЗАЩИТЕ»

Зав. кафедрой ИиИТ: **к.т.н. Е.В. Булатников**
(Подпись)

Москва

2026

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ



**Факультет Информационных технологий
кафедра Информатики и информационных технологий**

направление подготовки 09.03.02 «Информационные системы и технологии»

УТВЕРЖДАЮ

Зав. каф., к.т.н. Е.В. Булатников

« ____ » _____ 2026 г. _____

(Подпись)

ЗАДАНИЕ НА БАКАЛАВРСКУЮ РАБОТУ

Студент Гайдарь Максим Денисович группа 221-3711
(Фамилия, Имя, Отчество)

Тема Разработка сайта для настольно-ролевой игры

Утверждена приказом по Университету № 7174-УД от «30» декабря 2025 г.

1. Срок представления работы к защите « 09 » июня 2026 г.

2. Исходные данные для выполнения работы: Техническое задание, техническая литература по теме ВКР

3. Содержание бакалаврской работы: Введение, аналитическая часть, конструкторская часть, практическая часть, заключение

4. Перечень графического материала Презентация

5. Дата выдачи задания: «25» сентября 2025 г.

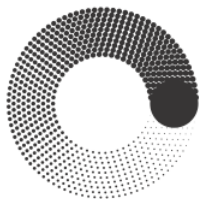
6. Руководитель: _____ /М.А. Кандрашина/

(Подпись) (И.О. Фамилия)

7. Задание к исполнению принял:  /М. Д. Гайдарь/

(Подпись) (И.О. Фамилия)

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ



**Факультет Информационных технологий
кафедра Информатики и информационных технологий**

направление подготовки 09.03.02 «Информационные системы и технологии»

КАЛЕНДАРНЫЙ ПЛАН

Студент Гайдарь Максим Денисович **группа** 221-3711
(Фамилия, Имя, Отчество)

Тема ВКР: Разработка сайта для настольно-ролевой игры

№ п/п	Наименование этапа ВКР	Срок выполнения этапа	Отметка о выполнении
1.	Исследование предметной области	26.09.2025-19.10.2025	Выполнено
2.	Разработка технического задания	20.10.2025-02.11.2025	Выполнено
3.	Эскизное проектирование	03.11.2025-23.11.2025	Выполнено
4.	Техническое проектирование	24.11.2025-21.12.2025	Выполнено
5.	Рабочее проектирование	22.12.2025-01.02.2026	Выполнено
6.	Разработка	02.02.2026-29.03.2026	Выполнено
7.	Тестирование	30.03.2026-26.04.2026	Выполнено
8.	Приемосдаточные испытания	27.04.2026-24.05.2026	Выполнено

Руководитель М.А. Кандрашина
(Фамилия И.О., степень, звание) (Подпись)

Зав. каф. ИиИТ /Е.В. Булатников/
(Подпись)

« » 2026 г.

Аннотация

В данной выпускной квалификационной работе рассматривается процесс разработки веб-платформы для авторской настольной ролевой системы E'Magios Core. Продукт реализован как одностраничное веб-приложение (SPA) на React и TypeScript по методологии Feature-Sliced Design и включает интерактивную базу данных игровых материалов, электронные книги правил, редактор персонажей, глобальный модуль бросков кубов, авторизацию с разграничением ролей, облачное хранение данных и аналитический дашборд.

Структура работы представлена введением, тремя главами, заключением, библиографическим списком и приложениями. Во введении обоснована актуальность темы. Первая глава посвящена анализу предметной области, сравнению аналогов и обоснованию выбора технологий. Во второй главе описано проектирование архитектуры, модели данных, доступа к данным и пользовательских интерфейсов. Третья глава посвящена программной реализации и тестированию. Заключение содержит итоги работы и перспективы развития.

Abstract

This final qualifying work examines the development of a web platform for the author's tabletop role-playing system E'Magios Core. The product is implemented as a single-page application (SPA) built with React and TypeScript following the Feature-Sliced Design methodology, and it includes an interactive database of game materials, electronic rulebooks, a character editor, a global dice-rolling module, role-based authentication, cloud data storage, and an analytical dashboard.

The work consists of an introduction, three chapters, a conclusion, a bibliography, and appendices. The introduction substantiates the relevance of the topic. The first chapter is devoted to the analysis of the subject area, the comparison of analogues, and the justification of the chosen technologies. The second chapter describes the design of the architecture, the data model, the data access layer, and the user interfaces. The third chapter covers the software implementation and testing. The conclusion contains the results of the work and the prospects for its further development.

Реферат

Название: Разработка веб-платформы для настольной ролевой системы.

Объём пояснительной записки составляет 92 страницы, включает 15 рисунков, 5 таблиц, 8 листингов и 3 приложения. При написании работы использовано 20 источников.

Ключевые слова и словосочетания: веб-платформа, одностраничное приложение, React, TypeScript, Vite, Feature-Sliced Design, Firebase, Firestore, IndexedDB, настольная ролевая система, база данных, редактор персонажей.

Объект разработки — веб-платформа для цифрового сопровождения авторской настольной ролевой системы E'Magios Core. Цель работы — разработка многофункционального типобезопасного веб-приложения, обеспечивающего удобный доступ к игровым материалам и предоставляющего интерактивные инструменты для подготовки и ведения игрового процесса. В работе применены методы компонентного проектирования пользовательских интерфейсов, слоистой архитектуры Feature-Sliced Design, декларативного описания доменных сущностей и модульного автоматизированного тестирования. Результатом работы является действующая веб-платформа с разделяемым набором UI-примитивов, изолированным слоем доступа к данным и покрытыми тестами расчётами. Область применения результатов — цифровая поддержка настольных ролевых игр и авторских игровых систем. Перспективы развития связаны с расширением справочного контента, мобильной адаптацией и развитием средств сетевого взаимодействия игроков.

СОДЕРЖАНИЕ

Введение.....	9
Глава 1. Аналитическая часть.....	12
1.1 Предметная область, цели и задачи	12
1.2 Анализ аналогичных решений.....	14
1.3 Обоснование выбора технологий	17
1.3.1 React и TypeScript.....	17
1.3.2 Vite и React Router.....	19
1.3.3 Методология Feature-Sliced Design	20
1.3.4 Firebase: Authentication и Cloud Firestore	22
1.3.5 IndexedDB и клиентское кэширование	22
1.3.6 Obsidian и автоматизация подготовки контента	23
1.3.7 Git, GitHub Pages и непрерывная интеграция	24
1.4 Сопоставление ранней и новой архитектуры проекта	24
1.5 Выводы по главе 1	26
Глава 2. Конструкторская часть	27
2.1 Общая информация о проекте и дизайн-документ.....	27
2.2 Архитектура Feature-Sliced Design.....	28
2.3 Проектирование модели данных	31
2.3.1 Доменные сущности и декларативные схемы	31
2.3.2 Облачное хранилище и статусная модель контента.....	33
2.3.3 Разграничение доступа на основе ролей	34
2.3.4 Клиентское кэширование и манифест версий	35
2.4 Проектирование доступа к данным.....	37
2.5 Проектирование пользовательского интерфейса и навигации	38

2.5.1 Дизайн-система	38
2.5.2 Навигация и каркас страниц	39
2.6 Проектирование конвейера подготовки контента	40
2.7 Принципы проектирования и нефункциональные требования....	41
2.8 Выводы по главе 2	42
Глава 3. Практическая часть	44
3.1 Организация проекта и сборка	44
3.2 Слой shared: технический фундамент	46
3.3 База знаний (компендиум)	48
3.4 Книги правил	53
3.5 Редактор персонажей	55
3.6 Глобальный виджет бросков кубов и интеграция с Discord.....	59
3.7 Сквозные связки: оверлей деталей и единый рендер текста	62
3.8 Авторизация и профиль	63
3.9 Дашборд данных	66
3.10 Главная страница и новости	68
3.11 Тестирование	69
3.12 Развёртывание и непрерывная интеграция	70
3.13 Выводы по главе 3	71
Заключение	73
Библиографический список	76
Приложение А. Техническое задание	78
Приложение Б. Презентация	81
Приложение В. Листинги программного кода.....	90

Введение

Актуальность разрабатываемого проекта обусловлена устойчивым ростом интереса к настольным ролевым играм и одновременным повышением требований к цифровым инструментам, сопровождающим игровой процесс. Современному пользователю уже недостаточно текстового описания правил: востребованы удобная навигация по материалам, быстрый поиск справочной информации, хранение данных персонажей, инструменты для бросков кубов и средства взаимодействия участников. Особенно это важно для авторских ролевых систем, где контент постоянно развивается и требует структурированного представления в единой цифровой среде.

Проект E'Magios Core представляет собой веб-платформу для одноимённой авторской настольной ролевой системы. Платформа объединяет несколько функциональных направлений: интерактивную базу данных игровых сущностей с гиперссылочной навигацией, электронные книги правил, редактор персонажей с облачным хранением, глобальный модуль бросков кубов с интеграцией Discord, авторизацию с разграничением ролей и аналитический дашборд. Дополнительной особенностью проекта является использование Obsidian как авторской среды хранения исходного контента с последующим автоматизированным преобразованием материалов для публикации.

Новизна работы заключается в том, что платформа объединяет в рамках единого типобезопасного одностраничного приложения два направления, которые в существующих решениях развиваются отдельно: крупный структурированный справочник правил и персональные интерактивные инструменты игрока. При этом приложение построено по строгой слоистой архитектуре Feature-Sliced Design, что обеспечивает предсказуемую организацию кода, переиспользование компонентов и тестируемость бизнес-логики — качества, нехарактерные для типовых любительских справочников по настольным играм.

Целью данной выпускной квалификационной работы является разработка многофункционального типобезопасного веб-приложения для поддержки настольной ролевой системы E'Magios Core, обеспечивающего удобный доступ к игровому контенту и предоставляющего пользователям интерактивные инструменты для подготовки и сопровождения игрового процесса.

Для достижения поставленной цели решаются следующие задачи:

- 1) Провести исследование предметной области цифровых инструментов для настольных ролевых систем и проанализировать существующие аналогичные веб-ресурсы, выявив их сильные и слабые стороны.
- 2) Обосновать выбор стека технологий и архитектурной методологии для реализации платформы.
- 3) Спроектировать архитектуру веб-приложения по методологии Feature-Sliced Design, включая модель данных, слой доступа к данным и пользовательские интерфейсы.
- 4) Спроектировать конвейер подготовки контента из авторской среды Obsidian.
- 5) Разработать разделяемую дизайн-систему пользовательских интерфейсов и реализовать функциональные модули: базу данных, книги правил, редактор персонажей, модуль бросков кубов, авторизацию и профиль, дашборд, главную страницу и новости.
- 6) Реализовать облачное хранение данных и разграничение доступа на основе ролей.
- 7) Обеспечить покрытие ключевой бизнес-логики модульными тестами и непрерывный контроль качества кода.
- 8) Провести тестирование для проверки корректности работы основных модулей и соответствия поставленным требованиям.

Проект реализуется на стеке React 18, TypeScript и Vite с применением сервисов Firebase для аутентификации и облачного хранения данных и публикуется как статическое приложение на GitHub Pages. Такой подход

обеспечивает практическую значимость разработки и создаёт основу для дальнейшего масштабирования цифровой экосистемы проекта E'Magios Core.

Пояснительная записка состоит из введения, трёх глав, заключения, библиографического списка и приложений. В первой, аналитической, главе исследуется предметная область, анализируются существующие аналоги и обосновывается выбор технологий. Во второй, конструкторской, главе описывается проектирование архитектуры, модели данных, доступа к данным, пользовательского интерфейса и конвейера подготовки контента. В третьей, практической, главе рассматривается программная реализация модулей платформы и их тестирование. Каждая глава завершается выводами. В заключении приводятся итоги работы и перспективы развития.

Глава 1. Аналитическая часть

1.1 Предметная область, цели и задачи

Предметной областью проекта являются цифровые инструменты сопровождения настольных ролевых игр. Настольная ролевая игра — это форма совместного повествования, в которой участники описывают действия своих персонажей, а результаты этих действий определяются правилами системы и бросками игровых костей. Как правило, один из участников выступает ведущим (мастером), который описывает игровой мир и управляет неигровыми персонажами, а остальные играют за своих персонажей. Игровой процесс опирается на значительный объём справочной информации: правила, описания заклинаний, эффектов, действий, навыков и предметов, а также на индивидуальные данные персонажей каждого игрока. Традиционно эта информация распределена между печатными книгами правил, рукописными листами персонажей и физическими игральными костями.

Система E'Magios Core является авторской: её правила, набор школ магии, заклинаний и механик разработаны самостоятельно и не повторяют существующие коммерческие системы. Это определяет специфику предметной области — отсутствие готовых справочников и инструментов, а также постоянное развитие правил, что делает создание собственной цифровой платформы не вспомогательной, а необходимой частью развития системы.

Перенос этих элементов в цифровую среду решает несколько практических проблем. Во-первых, структурированная база данных с поиском и фильтрацией ускоряет доступ к нужному правилу по сравнению с перелистыванием книги. Во-вторых, электронный лист персонажа автоматизирует расчёт производных характеристик и устраняет арифметические ошибки. В-третьих, цифровой модуль бросков обеспечивает прозрачность результатов и возможность их фиксации. В-четвёртых, облачное хранение делает данные доступными с разных устройств и защищает их от потери.

Для авторской системы, такой как E'Magios Core, цифровая платформа приобретает дополнительное значение. В отличие от устоявшихся коммерческих систем, правила авторской системы постоянно дорабатываются, и единый цифровой источник правды снимает проблему рассогласования версий правил между участниками. Платформа становится одновременно средством публикации правил, рабочим инструментом игрока и средством развития самой системы.

Целевая аудитория проекта включает две основные группы пользователей. Первая — игроки, которым необходим доступ к правилам, хранение персонажей и инструменты для бросков. Вторая — мастера (ведущие игры) и авторы системы, которым дополнительно требуется доступ к закрытым разделам, средства анализа данных и контроль над публикуемым контентом. Разделение этих ролей определяет требование к разграничению доступа.

Анализ предметной области показывает, что эффективная цифровая платформа для настольной ролевой системы должна удовлетворять ряду требований одновременно. Она должна обеспечивать структурированное хранение и быстрый поиск большого объёма взаимосвязанного справочного материала; предоставлять персональные инструменты, сохраняющие данные между сессиями и устройствами; связывать справочные данные с инструментами игрока, чтобы исключить дублирование и рассогласование; разграничивать доступ между обычными игроками и привилегированными пользователями; и при этом оставаться простой в сопровождении и развитии, поскольку авторская система постоянно дополняется. Совокупность этих требований и определяет постановку задачи разработки.

На основе выявленных потребностей можно сформулировать функциональные и нефункциональные требования к платформе. К функциональным относятся: ведение и отображение структурированной базы игровых сущностей с поиском, фильтрацией и сортировкой; отображение книг правил с навигацией и взаимными ссылками; создание, редактирование и

облачное хранение персонажей; выполнение бросков кубов с учётом характеристик персонажа и отправкой результатов во внешний сервис; авторизация пользователей и разграничение доступа по ролям; анализ данных платформы. К нефункциональным требованиям относятся сопровождаемость, тестируемость, производительность отклика интерфейса, безопасность доступа к данным и расширяемость, обеспечивающая добавление нового контента и функций без перестройки системы. Способы удовлетворения этих требований определяются на этапе проектирования и рассматриваются во второй главе.

На основании анализа предметной области сформулирована цель проекта — создание многофункционального типобезопасного веб-приложения для настольной ролевой системы E'Magios Core, объединяющего справочный контент, редактор персонажей, инструменты бросков и средства администрирования в единой цифровой платформе. Перечень решаемых задач приведён во введении и детализируется в последующих главах.

1.2 Анализ аналогичных решений

Для определения удачных подходов к организации контента, навигации и пользовательских инструментов был проведён анализ существующих веб-ресурсов, реализующих функции, схожие с разрабатываемой платформой. Анализ выполнялся по единому набору критериев, отражающих ключевые потребности целевой аудитории: наличие структурированной базы данных с фильтрацией, гиперссылочной навигации между сущностями, редактора персонажей, системы бросков, облачного хранения и разграничения доступа, интеграции с внешними сервисами, поддержки собственной авторской системы и качества архитектурной организации. Такой подход позволяет не только перечислить возможности каждого ресурса, но и системно сопоставить их с задачами разрабатываемой платформы. В качестве аналогов выбраны ресурсы dnd.su, ttg.club и Long Story Short, представляющие разные подходы к цифровой поддержке настольных ролевых игр.

Сайт **dnd.su** является русскоязычным онлайн-справочником по системе Dungeons & Dragons пятой редакции. Его сильная сторона — большой объём структурированного контента: классы, заклинания, черты, снаряжение, бестиарий. Для пользователя важны развитая система фильтрации, поиск по сущностям и гиперссылочная связность разделов. Ресурс демонстрирует значимость хорошо организованной базы данных и удобной навигации. Вместе с тем dnd.su ориентирован преимущественно на справочное наполнение, а интерактивные пользовательские инструменты не являются центральной частью платформы.

Проект **ttg.club** также является онлайн-справочником по Dungeons & Dragons и демонстрирует более современный подход к организации интерфейса. Помимо справочных разделов реализованы отдельные инструменты — калькулятор характеристик, токенатор. Особый интерес представляет удобная система поиска и фильтрации, позволяющая быстро отбирать объекты по параметрам. Однако ttg.club поддерживает уже существующую популярную систему, тогда как разрабатываемая платформа решает иную задачу — служит цифровой средой для собственной авторской системы и совмещает справочные материалы с пользовательскими модулями.

Сервис **Long Story Short** показывает другой вектор развития цифровых инструментов. Его основное назначение — интерактивный лист персонажа со встроенными формулами бросков и интеграцией результатов с внешними сервисами, включая Discord. Этот подход демонстрирует ценность персонализированных инструментов, сопровождающих игровой процесс. Для проекта E'Magios Core данный аналог важен как пример связывания редактора персонажа и системы бросков в единой среде. Вместе с тем Long Story Short сосредоточен на индивидуальном листе персонажа, тогда как разрабатываемая платформа дополнительно включает собственную базу данных и конвейер подготовки контента.

Помимо русскоязычных справочников, в качестве ориентира были рассмотрены крупные зарубежные платформы — D&D Beyond и Roll20. D&D

Beyond демонстрирует образец глубокой интеграции справочника с цифровым листом персонажа и автоматизацией правил, а Roll20 — пример виртуального игрового стола с инструментами ведущего. Эти платформы подтверждают общую тенденцию: ценность цифрового инструмента определяется не только полнотой справочника, но и связанностью справочных данных с персональными инструментами игрока. Вместе с тем обе платформы являются крупными коммерческими продуктами с закрытой кодовой базой и ориентированы на популярные системы, что не позволяет применить их для авторской системы напрямую и подтверждает обоснованность собственной разработки.

Результаты сравнительного анализа сведены в таблице 1.

Таблица 1 — Сравнение аналогов

Критерий	dnd.su	ttg.club	Long Story Short	E'Magios Core
База данных с фильтрами	+	+	—	+
Гиперссылочная навигация	+	+	—	+
Редактор персонажей	—	частично	+	+
Система бросков кубов	—	—	+	+
Облачное хранение и роли	—	—	+	+
Интеграция с Discord	—	—	+	+
Собственная авторская система	—	—	—	+
Типобезопасная архитектура одностраничного приложения	—	частично	—	+

Проведённый анализ показывает, что существующие проекты развиваются по одному из двух направлений: либо как крупные справочные базы правил, либо как набор персональных инструментов игрока. Разрабатываемая платформа E'Magios Core отличается тем, что объединяет оба подхода в рамках одного приложения, сохраняя уникальность как цифровая

среда для авторской системы. Кроме того, ни один из рассмотренных аналогов не решает задачу поддержки собственной, постоянно развивающейся системы правил: все они обслуживают зафиксированные коммерческие системы и не предполагают единого авторского источника контента с автоматизированной публикацией. Именно сочетание справочного и интерактивного функционала с конвейером подготовки контента из авторской среды и разграничением доступа отличает разрабатываемую платформу и определяет её практическую ценность для развития системы E'Magios Core.

По результатам анализа выделены следующие ориентиры для разработки:

- необходимость структурированной базы данных с удобной фильтрацией и быстрым поиском;
- важность гиперссылочной навигации между разделами и игровыми сущностями;
- целесообразность интеграции редактора персонажей и системы бросков кубов;
- полезность облачного хранения пользовательских данных и авторизации с разграничением ролей;
- перспективность объединения справочного и интерактивного функционала в рамках одного веб-ресурса.

1.3 Обоснование выбора технологий

В данном разделе описываются технологии, выбранные для реализации веб-платформы E'Magios Core, и обосновывается их применение. Ключевым отличием от ранней версии проекта является переход от набора статических HTML-страниц со скриптами к полноценному одностраничному приложению со строгой архитектурой и статической типизацией.

1.3.1 React и TypeScript

В качестве основной библиотеки построения пользовательского интерфейса выбран React 18. React реализует компонентный подход, при котором интерфейс описывается как композиция переиспользуемых

компонентов с собственным состоянием, а обновление представления выполняется декларативно при изменении данных [1]. При декларативном подходе разработчик описывает, как интерфейс должен выглядеть в зависимости от данных, а библиотека самостоятельно вычисляет минимальный набор изменений и применяет его к странице. Это решает характерную для ранней версии проекта проблему ручного императивного обновления элементов страницы, которое плохо масштабируется при росте числа интерактивных элементов, приводит к дублированию логики и трудноуловимым ошибкам рассинхронизации интерфейса и данных. В ранней версии каждый интерактивный участок требовал отдельного кода обновления; в компонентной модели достаточно изменить данные, а согласование отображения берёт на себя библиотека.

Язык TypeScript добавляет к JavaScript статическую типизацию [2] и компилируется в стандартный код ECMAScript [3], что сохраняет совместимость с любым современным браузером; фундаментальные возможности самого языка JavaScript подробно изложены в [4]. Для платформы со сложной доменной моделью — десятками типов игровых сущностей и структурой персонажа со множеством производных характеристик — типизация критически важна: она выявляет ошибки несоответствия данных на этапе компиляции, обеспечивает автодополнение и делает рефакторинг безопасным. Сочетание React и TypeScript является отраслевым стандартом разработки современных веб-приложений и обеспечивает баланс между скоростью разработки и надёжностью.

Состояние компонентов и побочные эффекты в React организуются с помощью механизма хуков. Хуки `useState` и `useReducer` хранят локальное состояние, `useEffect` выполняет побочные эффекты (загрузку данных, подписки), а `useContext` обеспечивает доступ к общему состоянию без передачи свойств через все промежуточные компоненты. В разрабатываемой платформе на основе хуков построены переиспользуемые единицы логики — собственные хуки `useAuth`, `useCompendiumData`, `useCharacterState` и другие,

которые инкапсулируют сложное поведение и применяются в разных частях приложения. Такой подход к организации логики невозможно было реализовать в ранней версии проекта на основе разрозненных скриптов, где состояние хранилось в глобальных переменных и неявно связывало модули между собой.

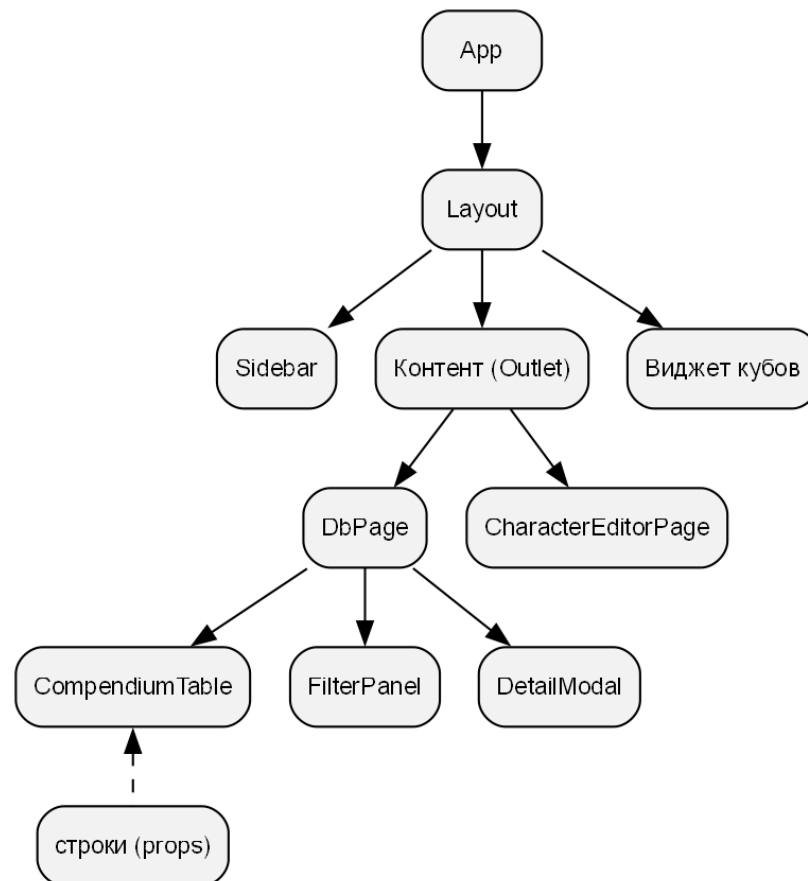


Рисунок 1.1 — Компонентная модель React — дерево компонентов и
однонаправленный поток данных от родителя к потомкам

1.3.2 Vite и React Router

Для сборки проекта и организации среды разработки выбран инструмент Vite. Он обеспечивает мгновенный запуск сервера разработки и горячую замену модулей (HMR), при которой изменения в коде отражаются в браузере без перезагрузки страницы и потери состояния, а для production-сборки выполняет оптимизацию и разбиение кода [5]. Это существенно ускоряет цикл разработки по сравнению с ручной организацией скриптов.

Помимо горячей замены модулей, Vite обеспечивает быстрый запуск среды разработки за счёт использования нативных модулей браузера во время

разработки и сборщика для production-режима. Это устраняет длительную предварительную сборку, характерную для более ранних инструментов, и сокращает время отклика при разработке до долей секунды.

Навигация между разделами одностраничного приложения реализована с помощью библиотеки React Router. Поскольку приложение публикуется на статическом хостинге GitHub Pages, который не поддерживает серверную обработку произвольных маршрутов, применяется режим хеш-маршрутизации (HashRouter): адрес раздела кодируется в части URL после символа «#», что гарантирует корректную работу прямых переходов и обновления страницы без серверной поддержки [6]. Маршрутизатор также обеспечивает вложенную маршрутизацию: общий каркас приложения описывается как родительский маршрут, а конкретные экраны — как дочерние, что позволяет единообразно подключать к ним боковую навигацию, виджет бросков и другие сквозные элементы.

1.3.3 Методология Feature-Sliced Design

Для организации структуры кода выбрана методология Feature-Sliced Design (FSD) — архитектурный подход, специально предназначенный для фронтенд-приложений [7]. FSD предписывает разделение кода на иерархию слоёв с однонаправленными зависимостями: модуль может импортировать только модули нижележащих слоёв. Это исключает циклические зависимости и делает архитектуру предсказуемой. Принцип однонаправленных зависимостей и отделения бизнес-логики от деталей реализации лежит в основе чистой архитектуры [8].

Выбор FSD обусловлен масштабом проекта. По мере роста числа функций плоская организация кода приводит к запутанным взаимным зависимостям, при которых изменение одного модуля непредсказуемо влияет на другие. Слоистая структура FSD локализует изменения, поощряет переиспользование общих компонентов и упрощает сопровождение. Детальное описание слоёв приведено в главе 2.

Помимо разделения на слои, методология вводит понятия среза (slice) и сегмента (segment). Срез — это вертикальное деление слоя по предметному признаку: например, в слое features срезами являются db, character-editor, dice, auth. Сегмент — это деление среза по техническому назначению: пользовательский интерфейс, модель данных, вспомогательная логика. Важным правилом является изоляция срезов одного слоя друг от друга: два среза слоя features не должны напрямую импортировать друг друга, а взаимодействуют только через нижележащие слои или через слой композиции. Это правило предотвращает скрытую связанность функций и позволяет развивать или заменять одну функцию, не затрагивая остальные. В разрабатываемой платформе соблюдение этих правил контролируется при разработке и проверяется на ревизии кода.

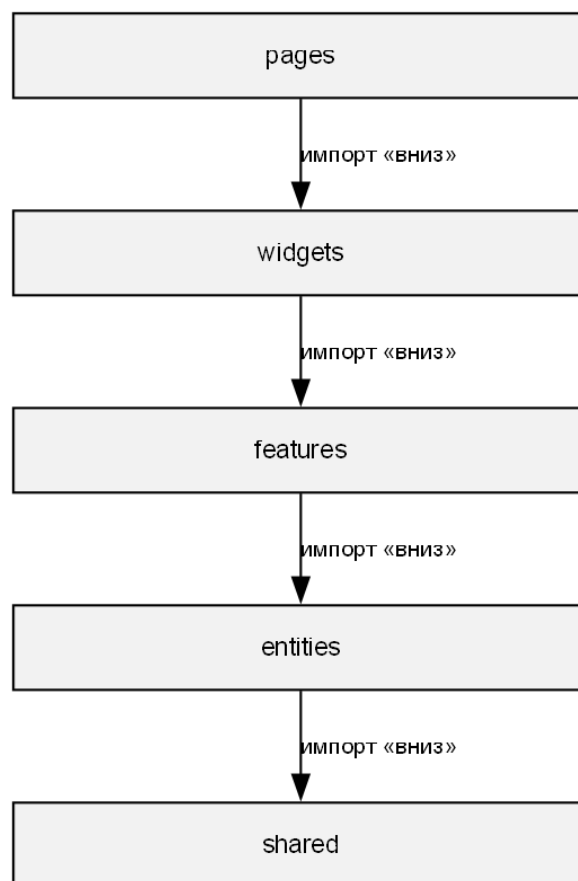


Рисунок 1.2 — Слои Feature-Sliced Design — shared, entities, features, widgets, pages с направлением импортов «сверху вниз»

1.3.4 Firebase: Authentication и Cloud Firestore

Для аутентификации пользователей и облачного хранения данных используется платформа Firebase. Сервис Firebase Authentication обеспечивает вход через аккаунт Google без необходимости разрабатывать и поддерживать собственную систему хранения паролей [9]. Облачная база данных Cloud Firestore предоставляет хранение структурированных документов с поддержкой работы в реальном времени и декларативными правилами безопасности [10].

Применение Firebase позволяет реализовать функции, требующие сервера, — регистрацию пользователей, хранение персонажей, разграничение доступа — без развёртывания собственного backend-сервера. Это согласуется с моделью публикации приложения как набора статических файлов и существенно снижает требования к инфраструктуре и сопровождению. Разграничение ролей реализовано через пользовательские утверждения (custom claims) в токене аутентификации: роль пользователя проверяется как на клиенте, так и в правилах доступа Firestore, что обеспечивает согласованность контроля доступа.

Важной чертой Cloud Firestore является декларативная модель безопасности: правила доступа описываются на стороне сервера и применяются ко всем запросам независимо от клиента. Это означает, что ограничение доступа не зависит от корректности клиентского кода — даже при попытке прямого обращения к базе в обход интерфейса сервер применит правила. Язык правил безопасности Firestore и их синтаксис описаны в официальной документации [11]. Такой подход устраняет принципиальный недостаток ранней версии проекта, где ограничение доступа к закрытым материалам выполнялось исключительно средствами интерфейса и не являлось настоящей защитой.

1.3.5 IndexedDB и клиентское кэширование

Для ускорения работы со справочными данными применяется браузерное хранилище IndexedDB через библиотеку-обёртку idb. IndexedDB

представляет собой полноценную клиентскую базу данных, способную хранить значительные объёмы структурированных данных, в отличие от ограниченного по объёму простого локального хранилища [12]. Библиотека `idb` предоставляет к нему удобный интерфейс на основе промисов, упрощающий асинхронную работу [13]. Справочные данные игровых сущностей объёмны, но изменяются редко, поэтому их кэширование на стороне клиента значительно сокращает время загрузки при повторных обращениях. Кэш сопровождается манифестом версий: при обновлении данных версия в манифесте изменяется, и клиент перезагружает только устаревшие наборы. Такой подход уменьшает сетевую нагрузку и обеспечивает быстрый отклик интерфейса, поскольку при повторном открытии раздела данные отображаются мгновенно из локального хранилища, а сетевой запрос выполняется лишь при наличии обновления. Программные интерфейсы браузера, используемые в проекте, соответствуют спецификациям, изложенным в справочнике MDN [14].

1.3.6 Obsidian и автоматизация подготовки контента

Исходные материалы ролевой системы ведутся в приложении Obsidian — среде для работы с заметками в формате Markdown, поддерживающей внутренние связи между документами [15]. Obsidian используется как авторская среда, в которой создаются и редактируются главы книг правил и описания игровых сущностей.

Выбор Markdown как формата исходных материалов обусловлен его простотой и долговечностью: текст остаётся читаемым без специальных средств, удобно версионизируется и не привязан к конкретному приложению. Внутренние связи между заметками (wikilinks) позволяют автору выстраивать сеть взаимосвязанных правил и сущностей ещё на этапе написания.

Для публикации материалы преобразуются Python-скриптами: выполняется разбор заметок в структурированные данные, преобразование Markdown в формат, пригодный для отображения, и разрешение внутренних ссылок Obsidian в ссылки платформы. Выбор Python для этого слоя обусловлен

удобством обработки текста, простотой написания служебных сценариев и возможностью быстро адаптировать парсеры под изменение структуры исходных заметок. Такой конвейер обеспечивает единый источник правды: автор работает в удобной среде, а приложение получает согласованные данные, что снижает риск рассогласования между текстом правил и интерактивной базой и существенно сокращает объём ручной работы при обновлении контента.

1.3.7 Git, GitHub Pages и непрерывная интеграция

Для контроля версий используется система Git, для размещения приложения — сервис GitHub Pages, публикующий статические файлы [16]. Git обеспечивает хранение полной истории изменений, ведение разработки в отдельных ветках и безопасное внесение доработок; выбор GitHub Pages обусловлен тем, что приложение построено как статический набор файлов и не требует серверного рендеринга, что упрощает публикацию и снижает требования к инфраструктуре.

Сборка и публикация автоматизированы средствами непрерывной интеграции и доставки (CI/CD): при изменении основной ветки средствами GitHub Actions [17] выполняется проверка качества кода и сборка приложения, после чего результат публикуется. В состав проверки качества входят статический анализ кода, проверка форматирования, проверка типов и запуск модульных тестов. Это гарантирует, что в эксплуатацию попадает только код, прошедший автоматический контроль, и формирует дисциплину разработки, при которой качество поддерживается непрерывно, а не проверяется однократно перед сдачей.

1.4 Сопоставление ранней и новой архитектуры проекта

Поскольку разрабатываемая платформа является развитием ранее существовавшей версии проекта, отдельного рассмотрения заслуживает сопоставление двух архитектурных подходов. Ранняя версия представляла собой набор статических HTML-страниц, связанных таблицами стилей и набором сценариев на языке JavaScript без статической типизации. Логика

приложения распределялась между сценариями, состояние хранилось в глобальных переменных, а взаимодействие модулей выстраивалось через неявные зависимости. Такой подход обеспечивал быстрый старт и простоту публикации, но плохо масштабировался: по мере роста числа функций усложнялось сопровождение, возрастал риск непреднамеренного влияния изменений одного раздела на другие, а отсутствие типизации и тестов повышало вероятность скрытых ошибок.

Новая версия сохраняет преимущества прежней (статическая публикация, отсутствие выделенного сервера приложения), но устраняет её недостатки за счёт строгой компонентной архитектуры, статической типизации, изоляции слоёв, выделенного слоя доступа к данным и автоматизированного тестирования. Ключевые различия двух версий приведены в таблице 2.

Таблица 2 — Сопоставление ранней и новой архитектуры

Аспект	Ранняя версия	Новая версия
Архитектура	набор HTML-страниц и сценариев	одностраничное приложение React по методологии FSD
Типизация	отсутствует (JavaScript)	статическая (TypeScript)
Организация кода	глобальные переменные, неявные связи	слои с однонаправленными зависимостями
Стили	глобальный CSS	CSS-модули и дизайн-токены
Доступ к данным	прямые обращения из страниц	слой репозитория и кэширование
Разграничение доступа	только на уровне интерфейса	роли и серверные правила Firestore
Кэширование данных	отсутствует	IndexedDB и манифест версий
Навигация и меню	дублирование в каждой странице	декларативное описание структуры

Тестирование	отсутствует	модульные тесты и контроль качества в CI
--------------	-------------	--

Сопоставление показывает, что переход на новую архитектуру носит не косметический, а принципиальный характер: он меняет способ организации кода и управления данными, повышая надёжность и сопровождаемость платформы. При этом миграция выполнялась с сохранением функциональности и проверкой бизнес-правил, что подробно рассматривается в практической части.

1.5 Выводы по главе 1

В аналитической части исследована предметная область цифровых инструментов для настольных ролевых систем и определена целевая аудитория проекта. Проведён сравнительный анализ аналогичных решений (dnd.su, ttg.club, Long Story Short), который показал, что существующие ресурсы развиваются либо как справочные базы, либо как персональные инструменты, и обосновал нишу разрабатываемой платформы, объединяющей оба подхода. На основе анализа сформулированы ориентиры для разработки и обоснован выбор технологического стека: React и TypeScript для типобезопасного компонентного интерфейса, Vite и React Router для сборки и навигации, методология Feature-Sliced Design для организации кода, Firebase для аутентификации и облачного хранения, IndexedDB для клиентского кэширования, конвейер на основе Obsidian и Python для подготовки контента, а также Git, GitHub Pages и CI/CD для публикации. Выбранные решения создают основу для проектирования, рассматриваемого во второй главе.

Глава 2. Конструкторская часть

2.1 Общая информация о проекте и дизайн-документ

Платформа E'Magios Core проектируется как одностраничное веб-приложение, предназначенное для цифрового сопровождения авторской настольной ролевой системы. Основная задача приложения — объединить справочный контент, персональные игровые инструменты и средства администрирования в рамках единой среды с предсказуемой архитектурой. В отличие от обычного статического сайта с текстовыми страницами, платформа сочетает навигацию по книгам правил, структурированную базу данных игровых сущностей, редактор персонажей, модуль бросков кубов, облачное хранение и разграничение доступа. Это определяет повышенные требования к организации кода: при таком объёме функциональности случайная организация быстро привела бы к запутанным зависимостям, поэтому архитектура спроектирована с опорой на строгую методологию ещё до начала реализации. Полный перечень требований к платформе оформлен в виде технического задания (приложение А).

С точки зрения пользователя приложение состоит из нескольких функциональных разделов: главной страницы, новостей, интерактивной базы данных игровых сущностей, электронных книг правил, редактора персонажей, страницы профиля и аналитического дашборда. Сквозными по отношению к разделам являются два механизма: глобальный виджет бросков кубов, доступный на любой странице, и глобальный оверлей детальной карточки сущности, открываемый по ссылке из любого текста. Доступ к части разделов и действий регулируется ролью пользователя.

Целевая аудитория платформы разделяется на две группы с разными потребностями. Рядовые игроки используют справочник, редактор персонажей и инструмент бросков; их сценарий — найти правило, подготовить персонажа, выполнить бросок в ходе партии. Ведущие и авторы системы дополнительно работают с закрытыми материалами, средствами анализа данных и контролем над публикуемым контентом. Это разделение

определяет состав ролей и гейтинг разделов: одни возможности доступны всем, другие — только авторизованным пользователям с соответствующей ролью.

С точки зрения пользовательской логики платформа состоит из нескольких взаимосвязанных подсистем: навигации и управления доступом, справочной базы данных, книг правил, редактора персонажей, модуля бросков кубов, профиля с настройками интеграций и аналитического дашборда. Эти подсистемы спроектированы как взаимодополняющие: пользователь может перейти от чтения правил к справочной карточке сущности, использовать её данные в редакторе персонажа, выполнить бросок с учётом характеристик и при необходимости отправить результат во внешний сервис. Именно связность подсистем, а не простое их соседство, является ключевой проектной особенностью платформы.

С технической точки зрения дизайн-документ фиксирует не только состав интерфейса, но и полный жизненный цикл данных: от авторского хранилища контента в Obsidian, через конвейер преобразования, в облачное хранилище и клиентский кэш, и далее в пользовательский интерфейс. Это разделяет редактирование контента и разработку приложения, что является осознанным архитектурным решением и позволяет развивать содержательную и программную части независимо.

2.2 Архитектура Feature-Sliced Design

Архитектура приложения построена по методологии Feature-Sliced Design, которая предписывает разделение кода на иерархию слоёв со строго однонаправленными зависимостями. Модуль вышележащего слоя может импортировать модули нижележащих слоёв, но не наоборот. Принципиальное различие между смежными слоями состоит в их назначении: слой `features` содержит пользовательские сценарии с бизнес-логикой (например, фильтрация базы или расчёт характеристик), а слой `widgets` — композиционные блоки интерфейса, собирающие фичи и сущности в законченные участки экрана; слой `entities` несёт доменные модели, а слой

shared — технические средства без привязки к предметной области. Состав слоёв приведён в таблице 3.

Таблица 3 — Слои Feature-Sliced Design

Слой	Назначение	Примеры в проекте
shared	Технический фундамент без бизнес-смысла	shared/ui, shared/cache, shared/repositories, shared/firebase, shared/telemetry
entities	Доменные модели и маппинг между объектом передачи данных и моделью	entities/compendium, entities/character, entities/content, entities/user
features	Пользовательские сценарии поверх сущностей	features/db, features/character-editor, features/dice, features/auth
widgets	Композиционные блоки интерфейса	widgets/db-table, widgets/sidebar, widgets/layout
pages	Экраны из виджетов и фич	pages/db, pages/character-editor, pages/book, pages/dashboard, pages/home

Порядок импортов «сверху вниз» (pages → widgets → features → entities → shared) исключает циклические зависимости и делает влияние изменений локальным и предсказуемым. Так, доменная модель сущности компендиума (слой entities) не зависит от того, как именно она отображается на странице (слой pages), поэтому изменение представления не затрагивает модель данных. Диаграмма слоёв и пример распределения модулей приведены на рисунке 2.1.

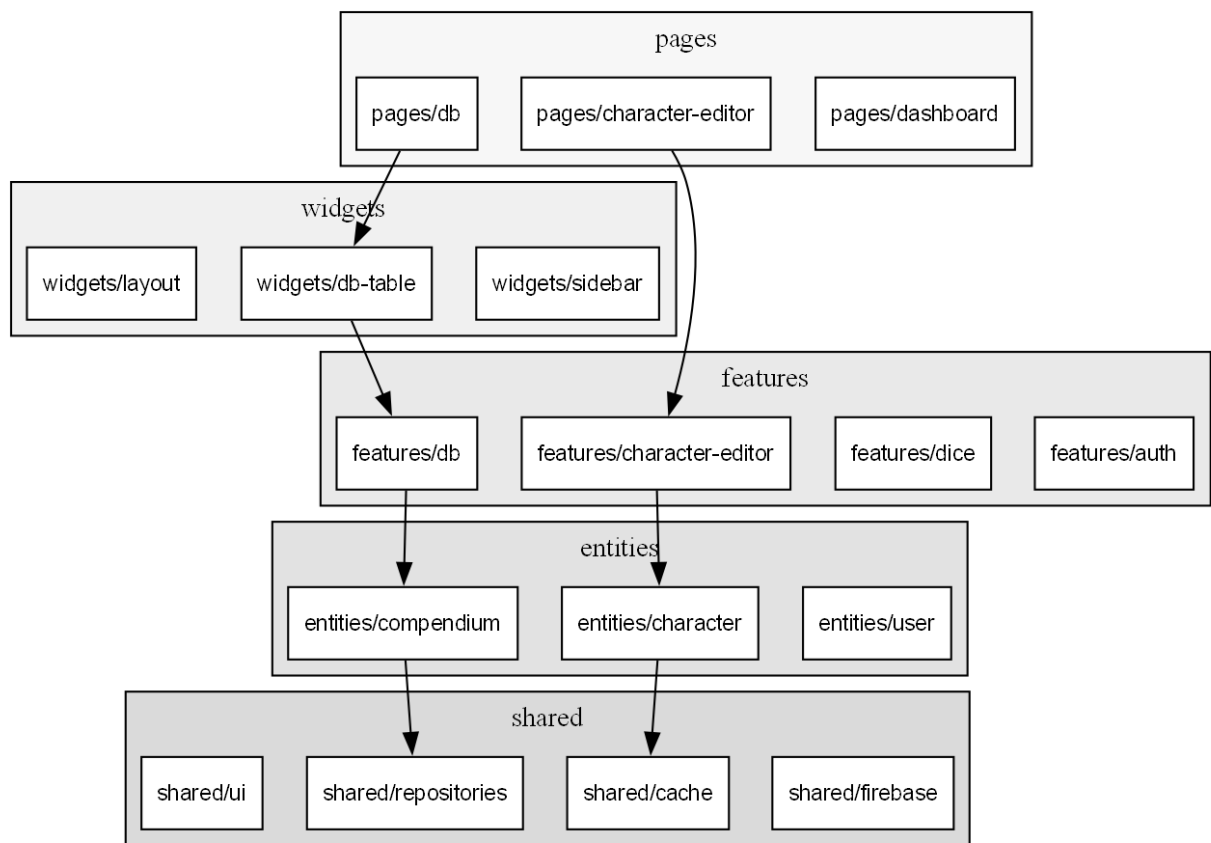


Рисунок 2.1 — Диаграмма слоёв FSD с примерами срезов и направлением зависимостей ($\text{pages} \rightarrow \text{widgets} \rightarrow \text{features} \rightarrow \text{entities} \rightarrow \text{shared}$)

Главное преимущество такой организации проявляется при сопровождении и развитии. Поскольку зависимости направлены строго вниз, а срезы одного слоя изолированы, разработчик, изменяющий конкретную функцию, может быть уверен, что его правки не затронут несвязанные части приложения. Новая функция добавляется как новый срез слоя `features`, не требуя изменения существующих; новый экран собирается на слое `pages` из готовых виджетов и фич. Это качественно отличается от ранней версии проекта, где логика была распределена между скриптами с неявными взаимными зависимостями через глобальные переменные, и любое изменение требовало проверки всего приложения.

Корневая сборка приложения выполняется в слое приложения: подключается провайдер аутентификации, настраивается маршрутизация и общий каркас страниц. Слой приложения играет роль композиционного уровня, на котором отдельные части соединяются в работающее целое, но сам

он не содержит бизнес-логики. Маршруты приложения, защищённые и публичные, объявляются декларативно, что иллюстрируется в практической части (листинг 1).

Спроектированная архитектура хорошо приспособлена к росту: новые экраны и функции добавляются как новые срезы соответствующих слоёв, переиспользуя уже имеющиеся примитивы интерфейса, репозитории и доменные модели. Поскольку зависимости направлены строго вниз, добавление функции на верхних слоях не требует изменения нижних, а значит, не создаёт риска регрессий в уже работающих частях. Эта предсказуемость особенно ценна для проекта, развитие которого планируется продолжать после защиты работы: заложенная структура задаёт ясные правила размещения нового кода и тем самым предотвращает постепенную деградацию архитектуры, характерную для проектов без чётких архитектурных границ.

2.3 Проектирование модели данных

Модель данных проектировалась с опорой на статическую типизацию: каждая доменная сущность описывается типом TypeScript, а соответствие данных этим типам проверяется компилятором. Это особенно важно для проекта со сложной предметной областью, где число типов сущностей велико, а их поля разнообразны: ошибка в имени или типе поля при работе с нетипизированными данными проявилась бы только во время выполнения и в неочевидном месте, тогда как при статической типизации она обнаруживается компилятором немедленно. Типизация также служит формой документации: по описанию типа сущности видно, какие поля она содержит и какого они типа, что облегчает понимание модели данных новым разработчиком.

2.3.1 Доменные сущности и декларативные схемы

Справочная часть платформы охватывает четырнадцать категорий игровых сущностей: заклинания, школы магии, эффекты, действия, навыки, архетипы, базовые статьи, типы действий, боевые компоненты, компоненты крафта, профессии, специализации, типы рецептов и рецепты. Эти категории

отражают структуру самой ролевой системы: заклинания принадлежат школам магии, накладывают эффекты и используют действия определённого типа; рецепты относятся к профессиям и специализациям и состоят из компонентов крафта. Таким образом, доменная модель не является произвольным набором таблиц, а воспроизводит смысловые связи предметной области, что и делает возможной гиперссылочную навигацию между сущностями. Для каждой категории в слое `entities/compendium` заданы три элемента: тип доменной модели, объект передачи данных (DTO), описывающий форму хранения, и функция-маппер, преобразующая DTO в доменную модель.

Разделение DTO и доменной модели введено сознательно. Объект передачи данных отражает форму хранения документа в облачной базе и содержит служебные поля — статус публикации, версию и дату изменения. Доменная модель, с которой работает интерфейс, очищена от служебных полей и содержит только смысловые атрибуты сущности. Маппер выполняет преобразование между этими представлениями и является единственным местом, где знание о форме хранения проникает в приложение. Благодаря этому изменение формата хранения (например, переименование поля в базе) затрагивает только маппер, а не весь код, использующий сущность.

Чтобы не описывать интерфейс отображения отдельно для каждой категории, состав колонок, фильтров и полей детальной карточки вынесен в декларативные схемы (`schema.ts`). На основании схемы виджеты строят таблицу, панель фильтров и детальную карточку единообразно для всех категорий. Такой подход делает справочник расширяемым: добавление новой категории сводится к описанию её типа, маппера и схемы, без изменения логики отображения. Например, сущность «заклинание» описывается полями названия, школы магии, действия, типа действия, дистанции, цели, длительности, типа урона, источника, требования к уровню и текстового описания, тогда как сущность «рецепт» имеет иной набор полей — профессию, специализацию, перечень компонентов и результат.

Декларативная схема позволяет отобразить обе сущности единым механизмом, сохранив различия в их структуре. Перечень категорий и соответствующих коллекций приведён в таблице 4.

Таблица 4 — Категории компендиума и коллекции Firestore

Категория	Коллекция	Ключевые поля
Заклинания	spells	название, школа, действие, дистанция, урон
Школы магии	schools	название, описание, связанные школы
Эффекты	effects	название, тип, описание
Действия	actions	название, тип действия, стоимость
Навыки	skills	название, категория, описание
Архетипы	archetypes	название, школы, особенности
Базовые	basics	название, раздел, описание
Типы действий	action_types	название, описание
Боевые компоненты	combat_components	название, эффект
Компоненты крафта	craft_components	название, профессия
Профессии	craft_professions	название, специализации
Специализации	craft_specializations	название, профессия
Типы рецептов	recipe_types	название, описание
Рецепты	recipes	название, профессия, компоненты

2.3.2 Облачное хранилище и статусная модель контента

Данные платформы хранятся в облачной базе Cloud Firestore в виде коллекций документов. Справочные сущности размещаются в коллекциях по категориям; данные пользователей — в поддереве `users/{uid}`, где `{uid}` — идентификатор пользователя; персонажи — в подколлекции `users/{uid}/characters`. Такая иерархическая организация решает сразу две

задачи. Во-первых, она естественным образом отделяет данные одного пользователя от данных других: персонажи всегда вложены в поддереву их владельца, что упрощает правила доступа — достаточно проверить совпадение идентификатора владельца с идентификатором запрашивающего. Во-вторых, размещение справочных данных в отдельных коллекциях по категориям позволяет загружать только нужную категорию, не извлекая весь справочник целиком. Документ Firestore представляет собой набор полей и поддерживает вложенные структуры, что удобно для хранения сложных игровых сущностей с массивами связанных значений.

Контент имеет статусную модель жизненного цикла: документ может находиться в одном из состояний — черновик (*draft*), на проверке (*review*), опубликован (*published*) или в архиве (*archived*). Публично доступны для чтения только опубликованные документы; черновики и материалы на проверке видны лишь авторам и редакторам. Такая модель позволяет вести работу над новыми материалами, не публикуя их преждевременно, и обеспечивает редакторский контроль перед публикацией. Это особенно важно для развивающейся авторской системы, где правила постоянно дорабатываются и должны проходить проверку, прежде чем стать доступными игрокам. Эта модель спроектирована совместно с системой ролей и реализуется правилами безопасности Firestore.

2.3.3 Разграничение доступа на основе ролей

В платформе предусмотрены три роли с возрастающими полномочиями: автор (*author*), редактор (*editor*) и администратор (*admin*). Автор создаёт и дорабатывает контент в пределах своих черновиков; редактор отвечает за проверку и публикацию материалов; администратор обладает полным доступом и управляет назначением ролей. Роль хранится в пользовательском утверждении (*custom claim*) токена аутентификации, что обеспечивает единый источник истины для проверки доступа как на клиенте, так и на сервере (в правилах Firestore). Выбор хранения роли в токене, а не в отдельном документе, обусловлен тем, что токен проверяется при каждом запросе

автоматически, и серверные правила получают роль без дополнительного обращения к базе, что упрощает правила и исключает рассогласование между источником роли и проверкой доступа.

Спроектированный процесс работы с контентом выглядит так: автор создаёт документ в статусе «черновик» и может отправить его на проверку; редактор переводит материал из проверки в публикацию или возвращает в черновик, а также архивирует опубликованное; администратор обладает полным доступом. Каждый допустимый переход статуса жёстко закреплён за конкретной ролью, что исключает несанкционированную публикацию: автор не может опубликовать материал самостоятельно, минуя проверку редактором. Это формализует редакционный процесс и делает его частью модели доступа, а не соглашением между участниками.

Серверные правила безопасности являются основным рубежом контроля доступа, а интерфейсный гейтинг лишь скрывает недоступные действия. Разделение на два рубежа важно концептуально: интерфейсный гейтинг улучшает удобство (пользователь не видит недоступных кнопок), но не обеспечивает безопасность, поскольку клиентский код можно обойти; настоящую защиту даёт серверная проверка, выполняемая при каждом запросе независимо от клиента. Реализация правил приведена в практической части (листинг 8).

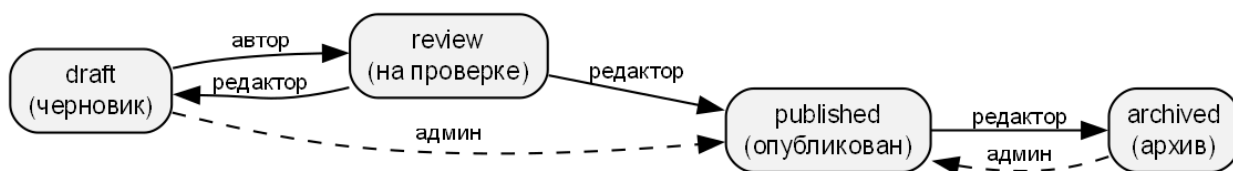


Рисунок 2.2 — Диаграмма состояний контента draft → review → published → archived с указанием ролей, выполняющих переходы

2.3.4 Клиентское кэширование и манифест версий

Поскольку справочные данные объёмны и изменяются редко, спроектирован механизм клиентского кэширования в IndexedDB. При первом обращении данные загружаются из облака и сохраняются в локальном хранилище вместе с номером версии. При повторных обращениях приложение

сначала отображает кэшированные данные, затем сверяет их версию с манифестом версий, публикуемым вместе с данными. Сетевой запрос выполняется только если версия в манифесте новее кэшированной.

Манифест версий представляет собой отдельный публично читаемый документ, в котором для каждой коллекции указан номер её текущей версии. При обновлении данных контентный конвейер увеличивает соответствующий номер. Благодаря этому клиент способен точно определить, какие именно наборы данных устарели, и перезагрузить только их, не трогая остальные. Такая схема сочетает мгновенный отклик (данные показываются из кэша немедленно) с гарантией актуальности (устаревшие данные обновляются в фоне). Проектное решение учитывает и граничный случай: пустой набор данных не считается достоверным кэшем, чтобы устаревшая пустая запись не закрепила в интерфейсе ложное состояние «ничего не найдено». Диаграмма этого процесса приведена на рисунке 2.3.

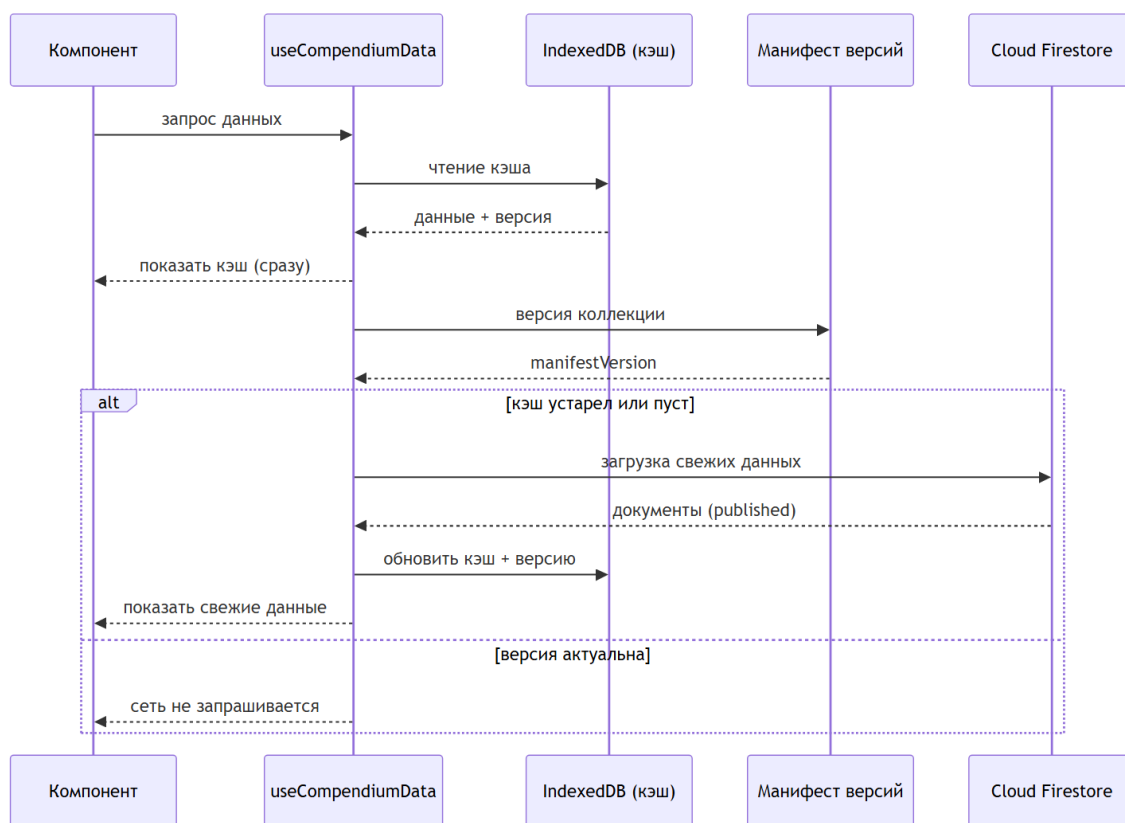


Рисунок 2.3 — Sequence-диаграмма загрузки данных — компонент → useCompendiumData → IndexedDB (кэш) → манифест версий → Firestore (при устаревании)

2.4 Проектирование доступа к данным

Доступ к данным изолирован в слое репозитория (shared/repositories). Компоненты интерфейса не обращаются к Firestore напрямую — они вызывают методы репозитория, который инкапсулирует детали запроса, фильтрацию по статусу публикации и преобразование документов в доменные модели. Это решает две задачи: централизует логику доступа и упрощает тестирование, поскольку репозиторий можно заменить тестовой реализацией.

Паттерн «репозиторий» [18] выбран сознательно как способ провести чёткую границу между предметной логикой и технологией хранения. Если в будущем потребуется сменить хранилище или добавить альтернативный источник данных, изменения затронут только реализацию репозитория, а весь использующий его код останется неизменным, поскольку зависит лишь от интерфейса методов, а не от их внутреннего устройства. Кроме того, сосредоточение всех обращений к базе в одном слое облегчает аудит безопасности и позволяет единообразно применять политику фильтрации по статусу публикации ко всем запросам.

Помимо инкапсуляции запросов, репозиторий выполняет фильтрацию по статусу публикации: для публичного чтения отбираются только опубликованные документы, что согласуется с серверными правилами безопасности. Импорт данных в облако выполняется отдельным служебным контуром с административными правами, минуя клиентские репозитории, поэтому клиентский код не содержит логики записи справочных данных и не может их изменить. Это чёткое разделение ответственности между чтением (клиент) и наполнением (служебный контур) повышает безопасность и упрощает рассуждение о потоке данных.

Над репозиториями расположен слой кэширования — хук useCompendiumData, реализующий описанную стратегию «сначала кэш, затем

сеть» и публикующий события телеметрии о попаданиях в кэш и результатах сетевых запросов. Таким образом, путь данных от хранилища к интерфейсу состоит из трёх отчётливых уровней: репозиторий (доступ), кэш (хранение и версионирование), компонент (отображение). Sequence-диаграмма взаимодействия этих уровней приведена на рисунке 2.3.

2.5 Проектирование пользовательского интерфейса и навигации

2.5.1 Дизайн-система

В основе интерфейса лежит разделяемая дизайн-система — набор UI-примитивов в `shared/ui`: кнопки, поля ввода, выпадающие списки, карточки, вкладки, таблицы, модальные окна, значки, индикаторы загрузки, всплывающие подсказки и заглушки пустого состояния. Все примитивы построены на единых дизайн-токенах — переменных тёмной темы, задающих цвета, отступы, скругления и размеры. Запрет на ввод произвольных цветов помимо токенов обеспечивает визуальную согласованность всех экранов.

Дизайн-токены организованы по группам: фоновые цвета (`--bg-*`), цвета текста (`--text-*`), акцентные цвета (`--accent-*`), отступы (`--spacing-*`), скругления (`--radius-*`) и размеры элементов каркаса. Базовая палитра задаёт тёмную тему с основным фоном тёмно-серого оттенка и изумрудным акцентным цветом, дополненным синим и фиолетовым. Введён запрет на использование произвольных цветовых значений в компонентах: допускаются только ссылки на токены. Это правило гарантирует, что вся палитра управляется централизованно, а перекраска интерфейса или добавление светлой темы сводится к изменению значений токенов, без правки отдельных компонентов.

Стили компонентов оформлены по технологии CSS-модулей: каждый компонент сопровождается файлом стилей, имена классов в котором автоматически изолируются и не конфликтуют с другими компонентами. Это устраняет характерную для ранней версии проблему глобального пространства имён CSS, при которой стили одного раздела непреднамеренно влияли на другой. Преимущество такого подхода — консистентность и

скорость разработки: новые экраны собираются из готовых примитивов, а изменение токена применяется ко всему приложению одновременно.

2.5.2 Навигация и каркас страниц

Навигация реализована маршрутизатором в режиме хеш-маршрутизации. Общий каркас (Layout) включает боковую панель навигации, область контента, глобальный виджет бросков и кнопку прокрутки вверх. Боковая панель строится из декларативного описания структуры книг и разделов, что исключает дублирование разметки меню: при добавлении новой главы или книги достаточно изменить описание структуры в одном месте, и меню перестроится автоматически. Это решает проблему ранней версии, где меню дублировалось в каждой странице и его изменение требовало правки множества файлов.

Боковая панель также выполняет функцию ориентации пользователя в большом объёме материалов: текущая книга и глава определяются автоматически по адресу, активный раздел раскрывается, а соответствующий пункт подсвечивается. Часть пунктов навигации (дашборд, закрытые книги) отображается в зависимости от роли пользователя, что связывает навигацию с системой доступа.

Общий каркас дополнен сквозными элементами удобства, действующими на всех экранах: плавающей кнопкой виджета бросков, кнопкой быстрого возврата к началу страницы и кнопкой сброса доступа к закрытым материалам. Размещение этих элементов в каркасе, а не на отдельных страницах, гарантирует их единообразное присутствие и поведение во всём приложении и избавляет от дублирования. Каркас спроектирован адаптивным, чтобы основная область контента и боковая панель корректно располагались при разной ширине окна. Карта экранов и навигации приведена на рисунке 2.4.

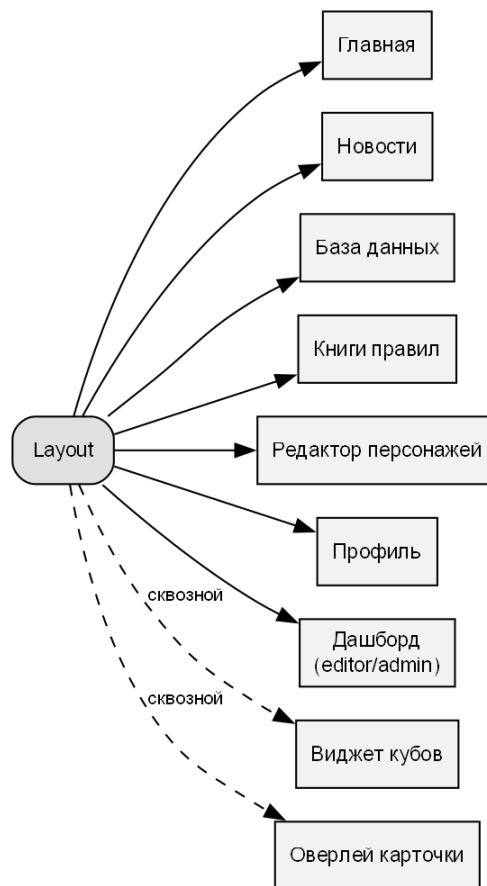


Рисунок 2.4 — Карта экранов и навигации приложения — главная, новости, БД, книги, редактор, профиль, дашборд, и сквозные виджеты

2.6 Проектирование конвейера подготовки контента

Конвейер подготовки контента спроектирован как отдельный от приложения слой. Исходные материалы ведутся в Obsidian Vault в формате Markdown. Python-скрипты выполняют их преобразование: парсеры разбирают заметки игровых сущностей в структурированные данные, конвертер преобразует главы книг в формат для отображения, а резолвер ссылок переводит внутренние ссылки Obsidian в ссылки платформы. Результат загружается в облачное хранилище (для справочных данных) или размещается как статические ассеты (для книг правил).

Для разных типов сущностей применяются отдельные парсеры, каждый из которых ориентирован на структуру конкретного типа заметок: парсер заклинаний извлекает название, действие, дистанцию, цель, длительность, школу и описание, а парсеры школ, эффектов и рецептов — соответствующие их роли наборы полей. Резолвер ссылок выступает общим слоем: там, где

требуется чистый текст, он удаляет служебную разметку, а там, где связь должна быть кликабельной, — формирует ссылку на карточку базы данных или на раздел правил. Благодаря этому структурированные данные пригодны не только для отображения, но и для навигации между связанными объектами. Схема конвейера приведена на рисунке 2.5.

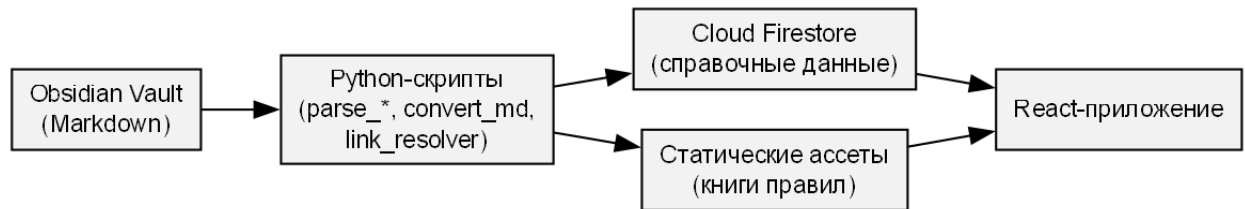


Рисунок 2.5 — Конвейер контента — Obsidian Vault (Markdown) → Python-скрипты (парсинг, конвертация, резолв ссылок) → Firestore / статические ассеты → React-приложение

Такой конвейер обеспечивает единый источник правды: автор работает в удобной среде, а приложение получает согласованные данные. Обновление данных выполняется через скрипты, а не ручным редактированием, что снижает риск рассогласования между текстом правил и интерактивной базой. Принцип единого источника правды особенно важен для проекта, в котором одни и те же игровые сущности используются в нескольких местах — в справочнике, в редакторе персонажей и в тексте книг. Если бы каждое из этих мест хранило собственную копию данных, их рассинхронизация была бы неизбежной; конвейер же гарантирует, что все потребители получают данные из одного авторского источника, а изменение правила в Obsidian после прогона скриптов отражается во всех разделах приложения одновременно.

2.7 Принципы проектирования и нефункциональные требования

Помимо функциональных возможностей, при проектировании платформы учитывались нефункциональные требования, определяющие качество программного продукта. Архитектурные решения подбирались так, чтобы каждое из этих требований было обеспечено системно, а не достигалось разовыми мерами.

Сопровождаемость обеспечивается слоистой архитектурой с однонаправленными зависимостями и изоляцией срезов: изменения локализуются, а структура кода самодокументируется расположением файлов по слоям. Этому же служат статическая типизация, единый набор UI-примитивов и вынесение доступа к данным в репозитории.

Тестируемость достигается выделением бизнес-логики в чистые функции и модули, не зависящие от интерфейса и внешнего состояния. Расчёты персонажа, парсер бросков и метрики аналитики реализованы как детерминированные функции, что позволяет покрывать их модульными тестами и проверять в конвейере непрерывной интеграции.

Производительность на стороне клиента обеспечивается кэшированием справочных данных в браузере с версионированием, параллельной загрузкой наборов данных и выполнением фильтрации и сортировки над данными в памяти. Сборка приложения оптимизируется и разбивается на части для ускорения первичной загрузки.

Безопасность реализуется на стороне сервера декларативными правилами доступа Firestore, проверяющими каждый запрос независимо от клиента, и хранением роли пользователя в проверяемом токене. Интерфейсный гейтинг рассматривается лишь как средство удобства, а не как рубеж защиты. Отображение внешнего содержимого книг выполняется с санитайзингом.

Расширяемость обеспечивается декларативным описанием сущностей, категорий справочника и структуры книг: добавление нового типа данных или раздела сводится к дополнению конфигурации, а не к изменению логики отображения. Заложенная архитектура позволяет добавлять новые функции как изолированные срезы, не затрагивая существующие.

2.8 Выводы по главе 2

В конструкторской части спроектирована архитектура платформы по методологии Feature-Sliced Design с пятью слоями и однонаправленными зависимостями. Спроектирована модель данных: четырнадцать категорий

справочных сущностей с декларативными схемами отображения, статусная модель жизненного цикла контента и разграничение доступа на основе трёх ролей с хранением роли в токене аутентификации. Спроектированы слой доступа к данным на основе паттерна «репозиторий» и механизм клиентского кэширования с манифестом версий. Спроектированы дизайн-система на единых токенах, навигация и каркас страниц, а также конвейер подготовки контента из Obsidian. Полученные проектные решения служат основой для программной реализации, описываемой в третьей главе.

Глава 3. Практическая часть

3.1 Организация проекта и сборка

Веб-приложение реализовано в каталоге `apps/web` и собирается инструментом Vite. Исходный код размещён в `src` и разделён по слоям Feature-Sliced Design: каталоги `app`, `pages`, `widgets`, `features`, `entities` и `shared` соответствуют слоям архитектуры, что делает структуру проекта самодокументируемой — по расположению файла можно судить о его назначении и допустимых зависимостях. Для импорта модулей настроен короткий псевдоним `@/`, указывающий на каталог `src`, что избавляет от длинных относительных путей и делает импорты устойчивыми к перемещению файлов.

Контроль качества кода автоматизирован составной командой `npm run check`, объединяющей четыре проверки: статический анализ кода (линтер ESLint) [19], проверку форматирования, проверку типов компилятором TypeScript и запуск модульных тестов. Эта команда выполняется при каждом изменении и в конвейере непрерывной интеграции: код, не прошедший хотя бы одну проверку, не попадает в основную ветку и не публикуется. Каждая из проверок решает свою задачу: линтер выявляет потенциально ошибочные конструкции и нарушения принятых правил, проверка форматирования поддерживает единый стиль оформления кода во всём проекте, проверка типов гарантирует согласованность данных и сигнатур функций, а тесты подтверждают корректность бизнес-логики. Совместное применение этих средств формирует многоуровневый барьер качества, при котором разные классы ошибок отсекаются на наиболее ранней возможной стадии — ещё до запуска приложения.

Корневая сборка приложения объявляет провайдер аутентификации, маршрутизацию в режиме хеш-адресации и общий каркас страниц. Объявление маршрутов приведено в листинге 1.

Листинг 1 — Объявление маршрутов и провайдеров приложения (App.tsx)

```

export function App() {
  return (
    <AuthProvider>
      <HashRouter>
        <TelemetryTracker />
        <Routes>
          <Route element={<Layout />}>
            <Route index element={<HomePage />} />
            <Route path="news" element={<NewsPage />} />
            <Route path="profile" element={<ProfilePage />} />
            <Route path="db" element={<DbPage />} />
            <Route path="character-editor" element={<CharacterEditorPage />} />
            <Route path="dashboard" element={<DashboardPage />} />
            <Route path=":bookKey/:chapterId" element={<BookPage />} />
            <Route path="*" element={<Navigate to="/" replace />} />
          </Route>
        </Routes>
      </HashRouter>
    </AuthProvider>
  );
}

```

Сборка приложения для эксплуатации выполняется в два этапа: проверка типов компилятором и сборка оптимизированного пакета инструментом Vite. На этапе сборки выполняются минификация кода, разбиение на части для ускорения загрузки и подготовка статических ассетов. Контент книг правил при сборке копируется в выходной каталог, что позволяет обслуживать его как статические файлы.

Публикация приложения выполняется на GitHub Pages: конвейер непрерывной доставки при изменении основной ветки запускает проверку качества и сборку, после чего публикует результат. Поскольку приложение использует хеш-маршрутизацию, прямые переходы и обновление страниц работают без серверной поддержки. Автоматизация публикации исключает ручные операции и связанные с ними ошибки, а условие прохождения проверки качества перед публикацией гарантирует, что в эксплуатацию не

попадёт код с ошибками типов, нарушениями стиля или непройденными тестами.

3.2 Слой shared: технический фундамент

Слой shared содержит технический фундамент без бизнес-смысла. Помимо дизайн-системы (shared/ui), в нём размещены клиент Firebase, репозитории доступа к данным, механизм кэширования, телеметрия и описание навигации.

Дизайн-система включает набор переиспользуемых примитивов: кнопку, модальное окно, карточку, поле ввода, выпадающий список, многострочное поле, вкладки, таблицу, значок, индикатор загрузки, всплывающую подсказку и заглушку пустого состояния. Каждый примитив реализован как отдельный компонент с собственным файлом стилей и единым стилем программного интерфейса, что делает их взаимозаменяемыми и предсказуемыми в использовании. Примитивы экспортируются через единую точку входа, поэтому подключение любого из них в произвольном месте приложения выполняется единообразно. Этот набор покрывает потребности всех экранов платформы и обеспечивает их визуальную согласованность.

Каждый примитив спроектирован так, чтобы инкапсулировать не только внешний вид, но и поведение, типичное для соответствующего элемента интерфейса. Например, компонент модального окна берёт на себя затемнение фона, закрытие по нажатию вне области и по клавише, блокировку прокрутки нижележащего содержимого; компонент таблицы поддерживает готовность к сортировке по колонкам; компонент заглушки пустого состояния единообразно отображает ситуацию отсутствия данных. Благодаря инкапсуляции поведения прикладные экраны не повторяют эту логику, а получают её готовой, что снижает объём кода и вероятность расхождений в поведении одинаковых по смыслу элементов на разных страницах. Введение полноценного набора примитивов было выполнено в первую очередь, поскольку единый визуальный язык повышает воспринимаемое качество продукта и ускоряет разработку всех последующих экранов.

Центральным элементом кэширования является хук `useCompendiumData`, реализующий стратегию «сначала кэш, затем сеть» с проверкой версии по манифесту. Его упрощённая логика приведена в листинге 2.

Листинг 2 — Стратегия «сначала кэш, затем сеть» (`useCompendiumData.ts`, упрощённо)

```
async function load() {
  // 1. Немедленно отдать данные из IndexedDB (пустые наборы игнорируем)
  const cached = await getCache<T>(cacheKey);
  if (cached && !isEmptyData(cached.data)) {
    setData(cached.data);
    setLoading(false);
  }
  // 2. Сверить версию кэша с манифестом
  const manifest = await ContentRepository.getManifest();
  const manifestVersion = manifest?.collections[manifestCollectionKey]?.version ??
  null;
  const cachedVersion = await getCacheVersion(cacheKey);
  // 3. Если версия актуальна – сеть не трогаем
  if (cached && manifestVersion !== null && cachedVersion >= manifestVersion) {
    logTelemetry({ eventName: 'cache_hit', route: cacheKey });
    return;
  }
  // 4. Иначе – загрузить свежие данные и обновить кэш
  const fresh = await fetcher();
  setData(fresh);
  await setCache(cacheKey, fresh, manifestVersion ?? cachedVersion + 1);
}
```

Особенностью реализации является трактовка пустого набора данных как промаха кэша: устаревшая пустая запись (например, закэшированная, пока данные ещё были черновиком) не должна закреплять в интерфейсе состояние «ничего не найдено» после публикации. Эта деталь была выявлена при тестировании и зафиксирована в коде.

В слое `shared` также размещён механизм телеметрии. При навигации между разделами автоматически фиксируется событие просмотра страницы с указанием маршрута и времени, проведённого на предыдущем экране;

загрузки данных фиксируют попадания в кэш и результаты сетевых запросов с измерением длительности. Собранные события используются служебным разделом для контроля поведения приложения в эксплуатации. Телеметрия реализована неинтрузивно: запись событий не блокирует основной поток и не влияет на отклик интерфейса, а её отказ не нарушает работу приложения. Наличие телеметрии отличает платформу от типового любительского справочника и обеспечивает наблюдаемость продукта.

Подключение к сервисам Firebase инициализируется в едином модуле клиента (`shared/firebase`). Инициализация выполняется однократно и переиспользуется всеми частями приложения, что исключает конфликт повторной инициализации при переходах между разделами — характерную проблему ранней версии, где подключение создавалось независимо в разных скриптах. Параметры подключения вынесены в конфигурацию и не содержат секретных значений: ключ конфигурации Firebase предназначен для клиента и определяет лишь адрес проекта, а реальный контроль доступа обеспечивают серверные правила.

Доступ к данным организован через три репозитория. Репозиторий компендиума отвечает за загрузку справочных категорий, репозиторий персонажей — за чтение и запись данных персонажей пользователя, репозиторий контента — за новости и манифест версий. Каждый репозиторий предоставляет узкий набор методов с понятными именами и скрывает детали обращения к облачной базе. Такое разделение по областям данных делает код доступа понятным и тестируемым: каждый репозиторий покрыт собственными тестами, проверяющими корректность выборки и преобразования данных. Компоненты интерфейса работают только с репозиториями и кэширующим хуком, что полностью изолирует их от деталей хранилища.

3.3 База знаний (компендиум)

База знаний реализована срезами `entities/compendium`, `features/db` и виджетом `widgets/db-table`. Доступ к данным инкапсулирован в репозитории

CompendiumRepository: для каждой категории определён метод, загружающий из соответствующей коллекции только опубликованные документы и преобразующий их в доменные модели. Общая функция выборки приведена в листинге 3.

Листинг 3 — Выборка опубликованных документов в репозитории
(CompendiumRepository.ts)

```
async function fetchPublished<T>(  
  collectionName: string,  
  mapper: (raw: Record<string, unknown>) => T,  
) : Promise<T[]> {  
  const col = collection(db, collectionName);  
  const q = query(col, where('status', '==', 'published'));  
  const snap = await getDocs(q);  
  return snap.docs.map((d) => mapper({ id: d.id, ...d.data() }));  
}
```

Категории описаны декларативно в конфигурации config.ts: для каждой заданы ключ, заголовок, имя коллекции, ключ манифеста, схема отображения и функция загрузки. Это позволяет странице базы данных обрабатывать все четырнадцать категорий единообразно — добавление новой категории сводится к дополнению конфигурации одной записью, без изменения кода страницы.

Детальная карточка сущности, открываемая при выборе записи, строится по той же схеме и состоит из нескольких секций: блока параметров, текстового описания и перечня связанных сущностей. Связи являются важной частью карточки: из заклинания можно перейти к его школе магии, из школы — к связанным школам и учебным заклинаниям, из описания эффекта — к соответствующему правилу. В описаниях выделяются выражения бросков, которые становятся кликабельными и выполняются через общий виджет. Для сохранения открытой карточки при переходах её состояние кодируется в адресе, что позволяет открыть конкретную карточку по прямой ссылке и пользоваться кнопками навигации браузера. При первой необходимости данные категории загружаются из облака. Чтобы сократить время ожидания,

загрузка отдельных наборов выполняется параллельно, поскольку браузер способен запрашивать несколько источников одновременно. Повторные запросы при переходах исключаются за счёт общего разделяемого результата загрузки и описанного механизма кэширования. При ошибке загрузки отдельного набора соответствующая часть данных заменяется пустым значением, что позволяет интерфейсу продолжать работу и не разрушает всю страницу из-за единичного сбоя.

Страница DbPage отображает категории в виде вкладок, реализует поиск, фильтрацию и сортировку, а также поддерживает прямые ссылки на состояние через параметры адреса (?tab=...&detail=...), что позволяет делиться ссылкой на конкретную карточку. Поскольку приложение не использует серверную базу данных, вся логика выборки, фильтрации и сортировки выполняется в браузере над уже загруженными данными. Для текущего объёма справочников это решение является достаточным, обеспечивает мгновенный отклик фильтров и упрощает публикацию проекта, не требуя серверной части. При значительном росте объёма данных подход допускает развитие без изменения архитектуры интерфейса: благодаря изоляции доступа к данным в репозиториях можно перейти к серверной фильтрации и постраничной загрузке, заменив реализацию репозитория, тогда как использующие его компоненты останутся неизменными. Таким образом, текущее клиентское решение является осознанным компромиссом, оптимальным для нынешнего масштаба и не создающим препятствий для будущего роста. Интерфейс базы знаний и детальной карточки показан на рисунках 3.1 и 3.2.

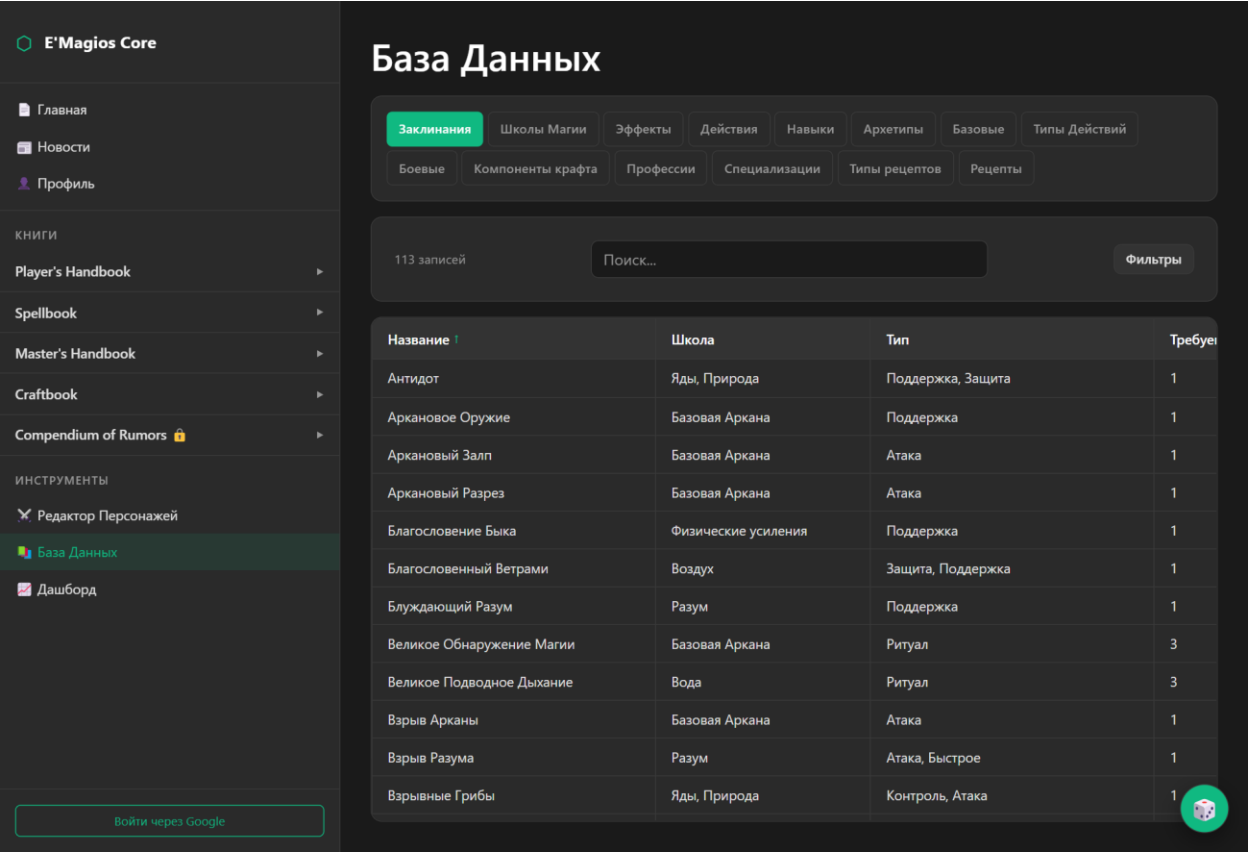


Рисунок 3.1 — Экран базы данных — вкладки категорий, поиск, фильтры, таблица

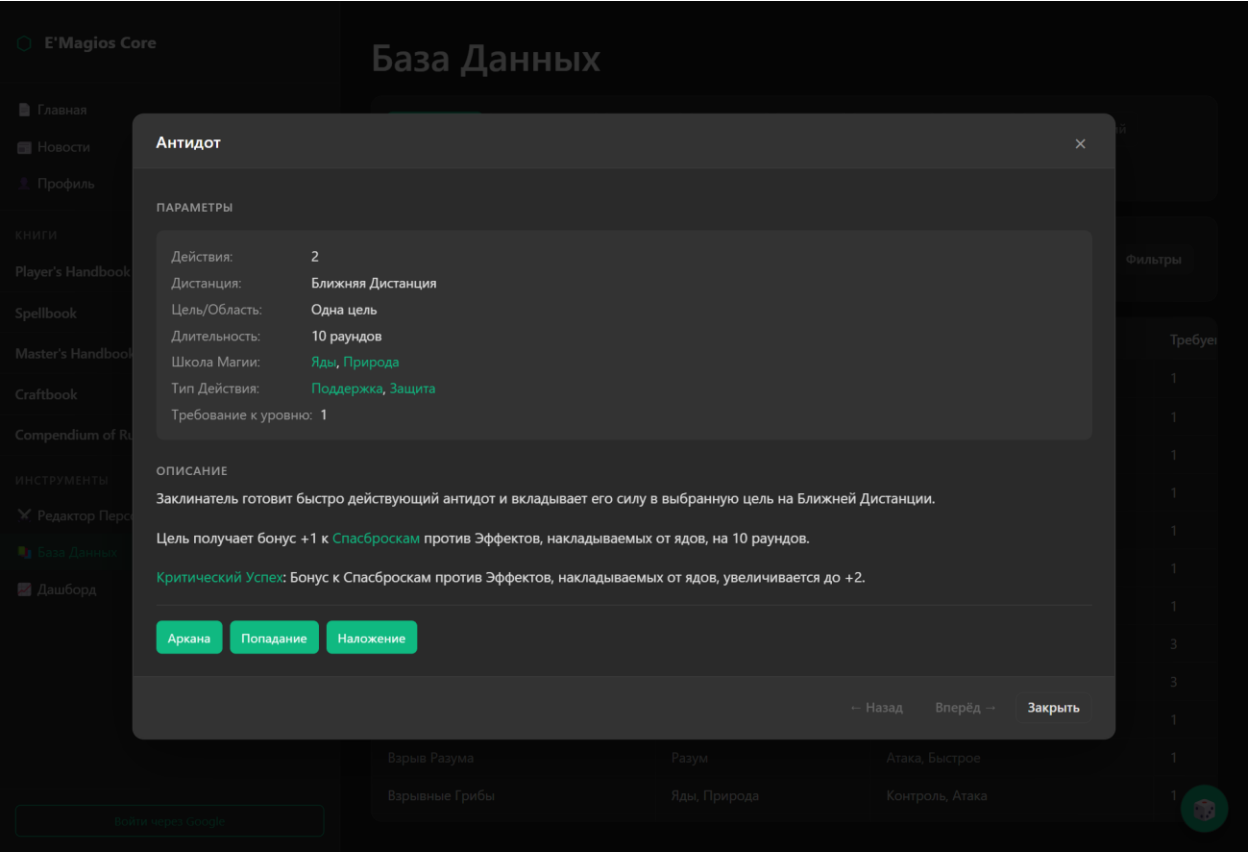


Рисунок 3.2 — Детальная карточка сущности с кросс-ссылками на связанные объекты

Фильтрация и сортировка реализованы в слое `features/db` и работают полностью на стороне клиента, так как все данные категории уже загружены в память. Фильтры формируются динамически на основании схемы: для каждого поля, помеченного как фильтруемое, строится набор доступных значений, извлечённый из самих данных. Пользователь выбирает значения во временном состоянии модального окна фильтров и применяет их явным действием — до нажатия кнопки применения таблица не перестраивается, что делает поведение предсказуемым и позволяет отменить ошибочный выбор. Поиск выполняется по нескольким текстовым полям одновременно (название, описание, тип, школа и другим), а сортировка поддерживает переключение направления по любой колонке. Поскольку поиск и сортировка работают над уже загруженными в память данными, результат обновляется мгновенно по мере ввода, без сетевых запросов и заметных задержек. Такая отзывчивость важна при использовании справочника непосредственно во время игры, когда нужно быстро найти нужное правило или заклинание. Текущее состояние вкладки, фильтров и открытой карточки кодируется в параметрах адреса, что обеспечивает воспроизводимость состояния по прямой ссылке и корректную работу кнопок «назад» и «вперёд» браузера.

Важной частью является разрешение кросс-ссылок: индекс сущностей (`useCompendiumIndex`) позволяет по имени связанного объекта открыть его карточку — из заклинания перейти к школе магии, из школы — к связанным заклинаниям. Индекс строится один раз после загрузки данных и сопоставляет имена сущностей с их идентификаторами и категориями, что делает разрешение ссылок операцией постоянной сложности. Это превращает базу из набора таблиц в навигационный слой над материалами системы, в котором пользователь изучает правила, переходя от конкретного заклинания к его школе, связанным эффектам и зависимым механикам.

3.4 Книги правил

Книги правил реализованы страницей `BookPage` и навигацией `shared/nav/books`. Структура книг (`phb`, `spellbook`, `master`, `craftbook`, `rumors`) и их глав задана декларативно, на её основе строятся боковое меню и заголовки. Содержимое глав хранится как статические ассеты и отображается с санитайзингом. Каждая книга описывается объектом с названием, признаком закрытого доступа и списком глав, а каждая глава — идентификатором, отображаемым названием и путём к файлу содержимого. Такое декларативное описание делает структуру книг централизованной: добавление новой главы или книги выполняется изменением одного описания, а не правкой множества страниц, что качественно отличается от ранней версии с дублированием меню в каждом файле.

Ключевая задача — перевод устаревших внутренних ссылок книг (например, `../spellbook/schools.html#fire`) в маршруты одностраничного приложения. Эту задачу решает чистая, покрытая тестами функция `resolveBookLink`, фрагмент которой приведён в листинге 4.

Листинг 4 — Преобразование внутренних ссылок книг в маршруты SPA
(`books.ts`, фрагмент)

```
export function resolveBookLink(rawHref: string, currentBookKey: string):  
ResolvedBookLink {  
  const href = rawHref.trim();  
  if (!href) return { kind: 'ignore' };  
  if (/^(https?:|mailto:|tel:)/i.test(href)) return { kind: 'external' };  
  if (href.startsWith('#')) return { kind: 'anchor', anchor: href.slice(1) };  
  // ... разбор пути и якоря ...  
  if (name === 'db') return { kind: 'route', route: `/db${query}`, anchor };  
  const bookKey = dir && dir in BOOKS ? dir : currentBookKey;  
  return { kind: 'route', route: `/${bookKey}/${name}`, anchor };  
}
```

Содержимое глав, перенесённое из ранней версии, представляет собой готовую HTML-разметку, которая отображается с санитайзингом — очисткой от потенциально небезопасных конструкций перед вставкой в страницу. Это

исключает выполнение произвольного кода из контента и является обязательной мерой при отображении внешнего HTML.

Ссылки внутри книг, ведущие на объекты базы данных, открывают детальную карточку, а внутрикнижные — выполняют переход к нужной главе и якорю. Преобразование ссылок необходимо потому, что в исходных материалах ссылки записаны в формате ранней версии (относительные пути к HTML-файлам), несовместимом с маршрутами одностраничного приложения. Книга rumors (сборник слухов, предназначенный для ведущего) помечена как закрытая и доступна только после ввода пароля; остальные книги открыты всем пользователям. Механизм ограничения доступа к книге реализован как переиспользуемый, поэтому закрытие или открытие книги сводится к изменению одного признака в описании навигации.

Ограничение доступа к закрытой книге реализовано на уровне интерфейса: при первом обращении пользователю показывается запрос пароля, а после успешного ввода признак доступа сохраняется в локальном хранилище браузера, чтобы не запрашивать пароль повторно при последующих открытиях. Предусмотрена и кнопка сброса этого признака, полезная при тестировании и демонстрации. В тексте работы важно зафиксировать, что данный механизм является именно интерфейсным удобством, а не средством защиты: содержимое книг физически доступно как статические файлы, поэтому пароль скрывает материалы в интерфейсе, но не заменяет серверное разграничение доступа. Для материалов, требующих настоящей защиты, в платформе предусмотрены серверные правила и роли. Экран книги правил показан на рисунке 3.3.

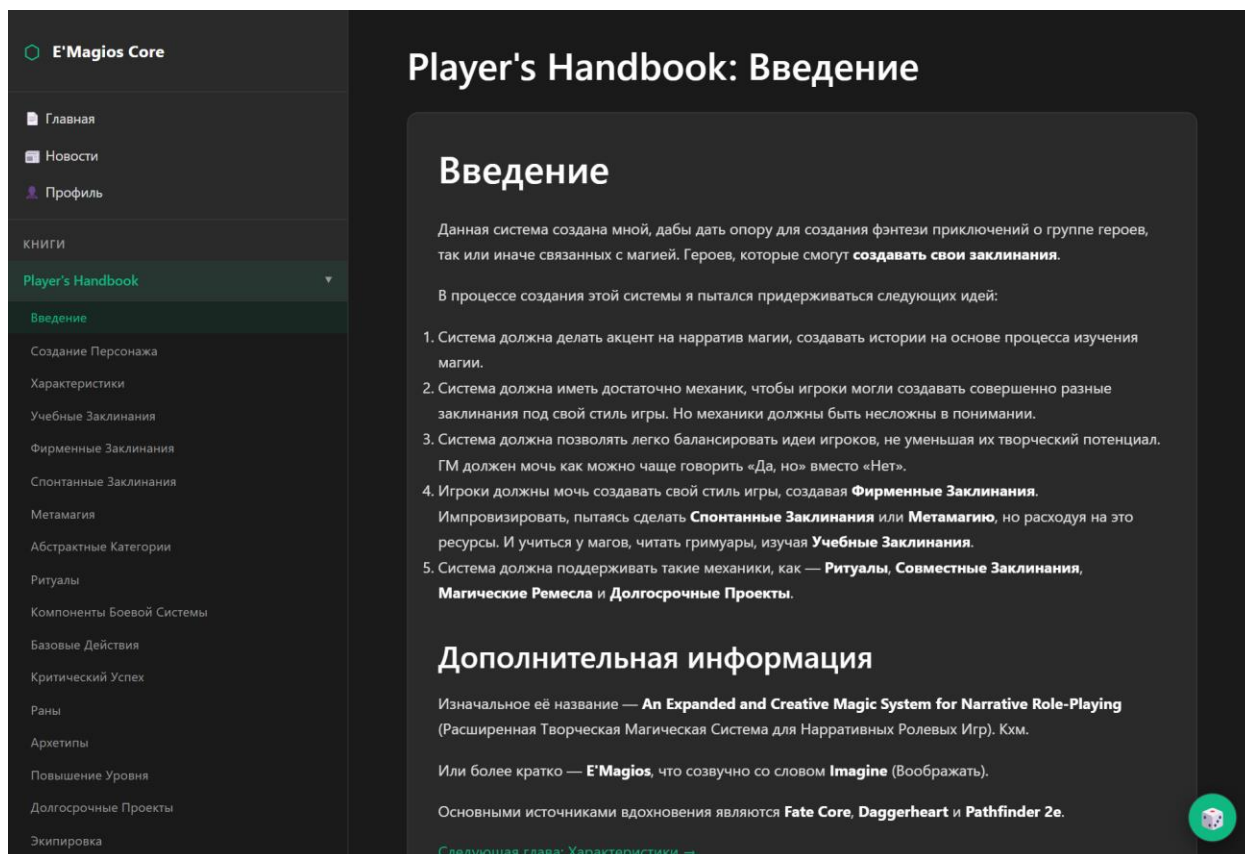


Рисунок 3.3 — Экран книги правил с боковой навигацией и кросс-ссылками

3.5 Редактор персонажей

Редактор персонажей (`features/character-editor`, `pages/character-editor`) — наиболее сложный по логике срез. Он объединяет форму ввода, расчёт производных характеристик, работу со школами и заклинаниями, облачное сохранение, импорт и экспорт в формате JSON. В отличие от справочных разделов, работающих с общими данными только на чтение, редактор оперирует персональными данными пользователя, поэтому его функциональность связана с авторизацией: создание и сохранение персонажа доступны только после входа. Состояние редактора — текущий пользователь, идентификатор редактируемого персонажа, список облачных персонажей, признак несохранённых изменений и таймер автосохранения — вынесено в отдельный хук, что отделяет управление состоянием от отображения формы и делает логику редактора тестируемой.

Производные характеристики персонажа вычисляются по табличным правилам системы в зависимости от уровня (1–20). Расчёт вынесен в чистую

функцию `calculateStats`, что позволило покрыть его модульными тестами и гарантировать соответствие правилам системы. Фрагмент реализации приведён в листинге 5.

Листинг 5 — Расчёт производных характеристик по уровню
(`characterCalculations.ts`, фрагмент)

```
const STAT_TABLE: Record<number, Omit<CharacterStats, 'level'>> = {
  1: { arcana: 1, health: 6, will: 4, speed: 6, initiative: 1,
    hitBonus: 1, effectBonus: 1, evasion: 10, fortitude: 10, actions: 2,
    reactions: 1 },
  // ... уровни 2-19 ...
  20: { arcana: 10, health: 25, will: 14, speed: 10, initiative: 10,
    hitBonus: 11, effectBonus: 11, evasion: 25, fortitude: 25, actions: 5,
    reactions: 3 },
};

export function calculateStats(level: number): CharacterStats {
  const clamped = Math.max(1, Math.min(20, level));
  const base = STAT_TABLE[clamped] ?? STAT_TABLE[1];
  return { level: clamped, ...base };
}
```

Производные характеристики не исчерпываются табличными значениями по уровню. Часть характеристик вычисляется по правилам системы на основе других: так, показатель внимательности определяется как восприятие, увеличенное на фиксированную величину, а характеристика стойкости имеет базовое значение и градации в зависимости от вложений игрока. Для прозрачности каждое производное значение сопровождается разбивкой, показывающей вклад базы по уровню, бонусов уровня, временных модификаторов и ручного переопределения. Пользователь может переопределить значение вручную с пометкой и последующим сбросом к расчётному, что важно для нестандартных игровых ситуаций. Такой подход заменяет несколько разрозненных модальных окон ранней версии единым прозрачным механизмом.

Разбивка характеристики реализована как переиспользуемый элемент: при наведении или нажатии на значок рядом с характеристикой раскрывается всплывающий блок с перечнем слагаемых итогового значения. Это решает важную для игрового инструмента задачу — игрок и ведущий должны понимать, как получено то или иное число, особенно при разборе спорных ситуаций за столом. Прозрачность расчётов повышает доверие к инструменту: вместо «магического» итогового значения пользователь видит обоснованную сумму, что соответствует природе настольной ролевой игры, где правила и расчёты открыты для всех участников.

Вынесение расчётов в чистые функции, не зависящие от интерфейса, позволяет с высокой уверенностью гарантировать корректность правил системы. Поскольку расчёт характеристик не обращается к внешнему состоянию и для одних и тех же входных данных всегда возвращает один и тот же результат, его поведение полностью описывается набором тестов, сверяющих результат с эталонными значениями для каждого уровня. Это критически важно при переносе игровых правил из ранней версии: любое непреднамеренное отклонение от исходной таблицы характеристик немедленно обнаруживается тестом, а не остаётся незамеченным до игровой партии.

Отдельного внимания заслуживает работа с заклинаниями персонажа. Заклинания делятся на учебные, фирменные и спонтанные, а также поддерживают вложенную структуру: к основному заклинанию могут относиться подзаклинания (сабспеллы), которые группируются с родителем и сворачиваются. Для каждого заклинания и подзаклинания в описании выделяются выражения бросков, превращаемые в кликабельные элементы. Выбор заклинаний и рецептов выполняется из общих справочников базы знаний через предпросмотр в детальной карточке, что устраняет дублирование игровых данных: при обновлении заклинания в исходных материалах изменения автоматически отражаются и в базе, и в редакторе.

Сохранение персонажа выполняется в подколлекцию `users/{uid}/characters` с автосохранением: изменения формы группируются и отправляются в облако с задержкой в три секунды, что снижает число записей и группирует последовательность действий пользователя. Перед сохранением редактор собирает данные формы, проверяет имя персонажа и наличие текущего пользователя, добавляет дату изменения и записывает документ; при наличии идентификатора документ обновляется, иначе создаётся новый. Для синхронизации списка персонажей между вкладками используется подписка на изменения в реальном времени (`onSnapshot`), благодаря которой пользователь видит актуальное состояние без ручной перезагрузки. Редактор также поддерживает экспорт и импорт персонажа в формате JSON, обеспечивая переносимость данных и возможность локальной работы вне облачной синхронизации. Экран редактора показан на рисунке 3.4.

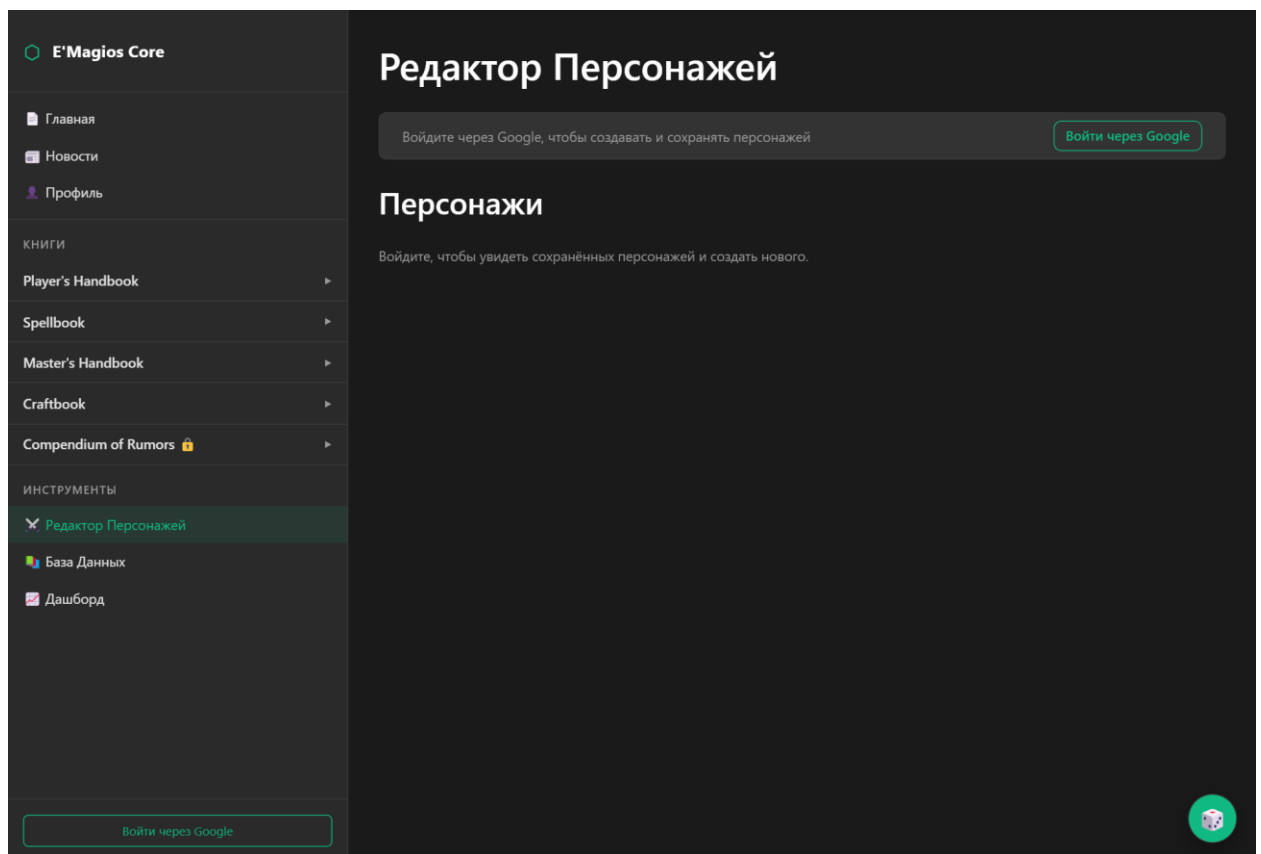


Рисунок 3.4 — Экран редактора персонажей — характеристики, школы, заклинания

3.6 Глобальный виджет бросков кубов и интеграция с Discord

Модуль бросков (`features/dice`) реализован как глобальный виджет, доступный на любой странице через общий каркас. Виджет включает историю бросков, быстрые кнопки стандартных кубов, поле команды `/roll`, выбор персонажа и настройки отображения. Виджет открывается плавающей кнопкой, присутствующей на всех экранах, благодаря чему бросок можно выполнить, не покидая текущий раздел — будь то чтение правил, просмотр карточки заклинания или работа с листом персонажа. Помимо быстрых кнопок для отдельных кубов, виджет содержит специализированные кнопки для характерных для системы бросков (Аркана, Попадание, Наложение), выполняемых двенадцатигранным кубом.

Ядром модуля является чистый парсер выражений бросков, портированный из ранней версии проекта и переработанный с инъекцией генератора случайных чисел для тестируемости. Парсер поддерживает сложные выражения вида `2d4+3d6-1`, масштабирование `1d12*2`, числовые модификаторы и ограниченный набор граней кубов. Фрагмент парсера приведён в листинге 6.

Листинг 6 — Парсер выражений броска (`rollExpression.ts`, фрагмент)

```
export function parseRollExpression(raw: string): ParsedRoll {
  const withoutCommand = raw.trim().toLowerCase().startsWith('/roll')
    ? raw.trim().slice(5).trim() : raw.trim();
  const normalized = withoutCommand.replace(/\s+/g, '');
  if (!/^[0-9dD+\-*/]+$/.test(normalized)) {
    throw new Error('Допускаются только цифры, d, +, -, * и /. Пример: /roll 2d4*2+3d6-1');
  }
  // ... разбиение на сегменты по + и - ...
  const m = text.match(/^(d*)(dD)(\d+)(?:([*/*])(d+))?$\/);
  const count = m[1] ? parseInt(m[1], 10) : 1;
  const sides = parseInt(m[2], 10);
  if (!ALLOWED_SIDES.includes(sides)) {
    throw new Error('Разрешены только D2, D4, D6, D8, D10, D12, D20 и D100.');
  }
}
```

```
// ...  
}
```

Парсер последовательно проверяет корректность ввода и при нарушении формата сообщает понятную ошибку — для пустой команды, недопустимого символа, двух знаков подряд или неподдерживаемой грани куба. Поддерживаются грани D2, D4, D6, D8, D10, D12, D20 и D100, а количество кубов и числовые модификаторы ограничены разумными пределами. Такая строгая проверка ввода предотвращает некорректные броски и делает поведение инструмента предсказуемым, а вынесение генератора случайных чисел в параметр позволяет проверять вычисление результата детерминированно в тестах.

После разбора выражения для каждого куба генерируется случайное значение, вычисляются промежуточные суммы сегментов и итоговый результат с учётом знаков, множителей и делителей. Модуль учитывает специфику системы: одиночный бросок d12 со значением 12 трактуется как критический успех, а со значением 1 — как критический провал; броски на наложение эффекта критов не имеют. Эти правила вынесены в отдельные предикаты и покрыты тестами.

При выборе персонажа к броскам применяются его бонусы, причём бонус зависит от типа броска: для бросков на попадание и на наложение используются разные характеристики листа. Разбивка бонусов по типу броска вынесена в отдельный модуль, что позволяет показать пользователю, из чего складывается итоговое значение. Связь с персонажем реализована через выбор сохранённого листа или через текущий открытый лист редактора, поэтому быстрые броски учитывают актуальные бонусы персонажа даже во время его редактирования.

История бросков хранит выполненные броски с их выражениями и результатами и доступна непосредственно в панели виджета, что позволяет вернуться к ранее выполненным броскам и при необходимости повторить их. Результат каждого броска сохраняется в историю, доступную в панели

виджета, и фиксируется как событие для последующей аналитики на дашборде; у авторизованных пользователей события дополнительно сохраняются в облаке. Формат события броска включает идентификатор пользователя, тип куба, выпавшее значение, модификатор, итог и контекст, что делает события пригодными для статистической обработки на дашборде — расчёта средних значений, отклонений и доли критических результатов по типам кубов и по пользователям. Если в профиле указан адрес вебхука Discord, результат броска дополнительно отправляется во внешний канал, что превращает локальный инструмент в средство публичных бросков для игровой группы. Отправка во внешний сервис выполняется по технологии вебхуков: приложение посылает на заданный пользователем адрес сообщение с результатом, оформленным согласно настройкам отправителя. Таким образом, один модуль обслуживает как личные броски игрока, так и публичные броски для всей группы, связывая платформу с внешней цифровой экосистемой. Виджет бросков показан на рисунке 3.5.

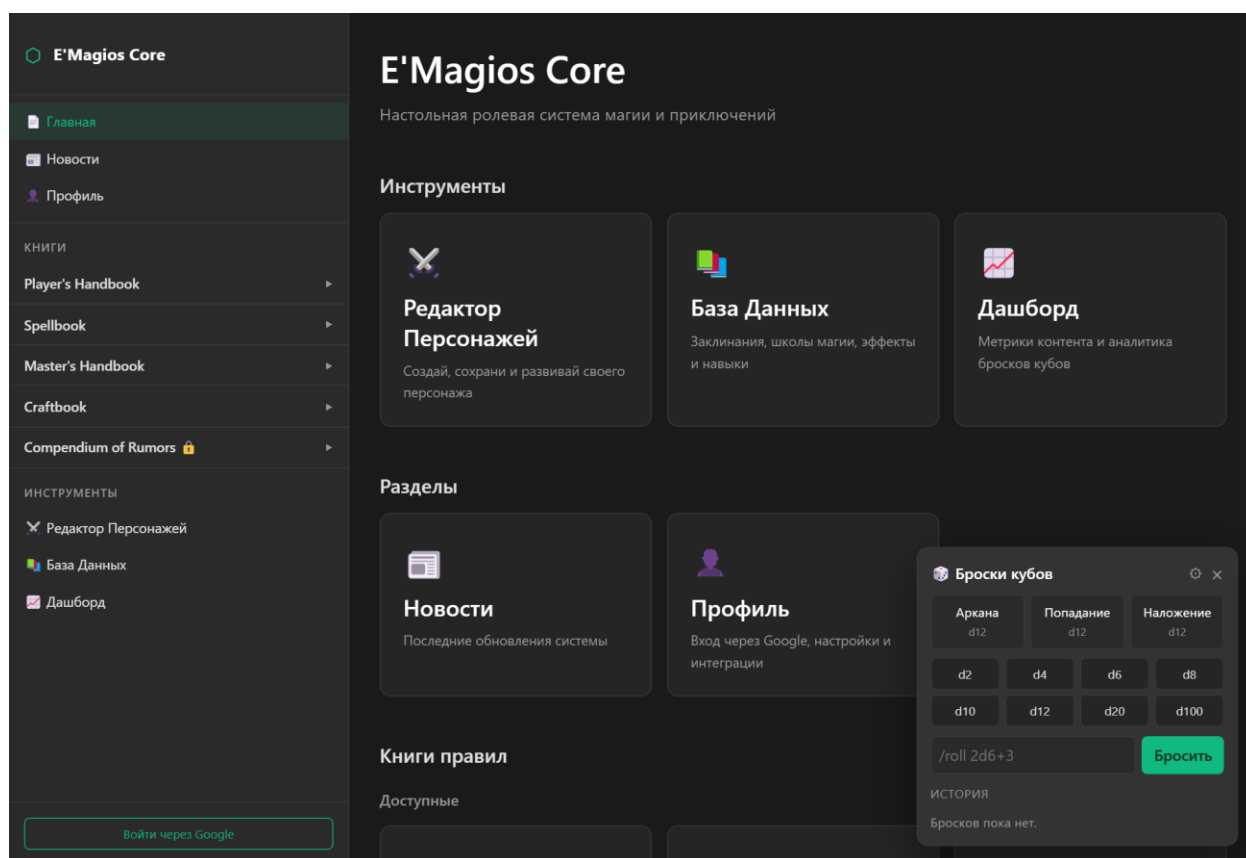


Рисунок 3.5 — Глобальный виджет бросков кубов — история, быстрые кубы, поле /roll, выбор персонажа

3.7 Сквозные связи: оверлей деталей и единый рендер текста

Для восстановления сквозных взаимодействий реализованы два механизма. Первый — глобальный оверлей детальной карточки (`shared/compendium/CompendiumOverlayContext`): клик по ссылке-объекту с любой страницы открывает карточку сущности поверх текущего экрана, без перехода на страницу базы данных. Поддерживается сквозная навигация вперёд-назад между связанными карточками. Для этого оверлей хранит стек просмотренных карточек: при переходе по ссылке внутри карточки текущая запись помещается в стек, а кнопки «назад» и «вперёд» перемещают пользователя по истории просмотра, подобно навигации в браузере. Один экземпляр оверлея обслуживает всё приложение, что исключает дублирование логики открытия карточек на разных экранах и гарантирует единообразное поведение. Данные для карточки загружаются лениво — только при открытии и только для нужной категории, что не нагружает страницу, с которой вызван оверлей.

Эти два механизма реализованы как общие контексты приложения и смонтированы на верхнем уровне, поэтому доступны из любого экрана. Необходимость в них возникла потому, что после разделения приложения на изолированные экраны сквозные взаимодействия ранней версии — открытие карточки объекта по ссылке из произвольного текста и единый виджет бросков — оказались разорванными. Глобальные контексты восстанавливают эти связи, не нарушая изоляцию срезов: экраны не обращаются друг к другу напрямую, а взаимодействуют через общий механизм нижележащего слоя.

Второй механизм — единый рендер текста (`shared/ui/LegacyText`): в описаниях сущностей, тексте книг и новостях формулы бросков превращаются в кликабельные элементы (открывают виджет бросков), а ссылки на объекты — в переходы к оверлею. Реализован единый конвейер обработки текста, который применяется во всех текстовых блоках приложения, благодаря чему формулы и ссылки ведут себя одинаково

независимо от того, где встречается текст — в карточке базы данных, главе книги или новости.

Технически конвейер обрабатывает текст в два прохода: сначала распознаёт выражения бросков по их характерному виду и оборачивает их в кликабельные элементы, затем выявляет ссылки на игровые объекты и связывает их с действием открытия оверлея. Распознавание выполняется по правилам, перенесённым из ранней версии, но собранным в одном месте, что устраняет прежнюю проблему, когда обработка ссылок и формул дублировалась в разных разделах с риском расхождения. Вынесение этой логики в общий компонент слоя `shared` означает, что любое улучшение обработки текста немедленно распространяется на все разделы, а новые текстовые блоки получают поддержку формул и ссылок без дополнительного кода.

3.8 Авторизация и профиль

Авторизация реализована срезом `features/auth` на основе `Firebase Authentication` со входом через `Google`. Вход через аккаунт `Google` выбран как способ, не требующий от пользователя создания и запоминания отдельного пароля и не возлагающий на проект ответственность за хранение учётных данных: проверку подлинности выполняет внешний провайдер, а приложение получает подтверждённый токен с идентификатором пользователя. При входе открывается всплывающее окно провайдера, после успешной аутентификации в котором приложение получает данные профиля — имя, адрес электронной почты и аватар. Провайдер `AuthProvider` подписывается на изменение состояния аутентификации единственным слушателем, а роль пользователя читает из пользовательского утверждения токена. Фрагмент провайдера приведён в листинге 7.

Листинг 7 — Чтение роли из токена и гард маршрута (`AuthContext.tsx`, `RequireRole.tsx`, фрагмент)

```
onAuthStateChanged(auth, (nextUser) => {  
  setUser(nextUser);
```

```

    setLoading(false);
    if (!nextUser) { setRole(null); return; }
    // Роль хранится в custom claim токена (ср. firestore.rules → userRole())
    nextUser.getIdTokenResult()
      .then((token) => setRole(normalizeRole(token.claims['role'])))
      .catch(() => setRole(null));
  });

  // Гард маршрута: пускает только при достаточной роли
  export function RequireRole({ require, children, redirectTo = '/' }:
RequireRoleProps) {
    const { uid, role, loading } = useAuth();
    if (loading) return <Spinner label="Проверка доступа..." />;
    if (!uid) return <Navigate to={redirectTo} replace />;
    const allowed = require === 'auth'
      || (require === 'editor' && (role === 'editor' || role === 'admin'))
      || (require === 'admin' && role === 'admin');
    return allowed ? <>{children}</> : <Navigate to={redirectTo} replace />;
  }

```

Гард `RequireRole` ограничивает доступ к защищённым маршрутам (дашборд, закрытые книги) на уровне интерфейса, тогда как основным рубежом контроля остаются серверные правила `Firestore`. Гард принимает требуемый уровень доступа и, пока состояние входа загружается, показывает индикатор; затем либо отображает защищённое содержимое, либо перенаправляет пользователя. Уровни доступа упорядочены: роль администратора включает права редактора, что позволяет описывать требования к маршруту лаконично. Фрагмент правил, реализующих процесс изменения статуса контента ролями, приведён в листинге 8.

Листинг 8 — Правила доступа к контенту по ролям и статусам (`firestore.rules`, фрагмент)

```

function canUpdateContent() {
  return request.auth != null &&
    (isAuthor() || isEditor() || isAdmin()) &&
    isValidStatus(resource.data.status) &&
    isValidStatus(request.resource.data.status) &&

```



```

(
  (isAuthor() && resource.data.status == 'draft'
    && request.resource.data.status in ['draft', 'review']) ||
  (isEditor() && (
    (resource.data.status == 'review'
      && request.resource.data.status in ['draft', 'published']) ||
    (resource.data.status == 'published'
      && request.resource.data.status == 'archived'))
  )) ||
  isAdmin()
);
}

```

Провайдер аутентификации реализован с единственным слушателем состояния входа, что исключает дублирование подписок и связанные с ним конфликты, характерные для ранней версии, где инициализация выполнялась независимо в нескольких модулях. Роль пользователя читается из токена один раз при входе и предоставляется всему приложению через контекст, что делает проверку доступа единообразной.

Страница профиля (`ProfilePage`) после входа отображает имя, электронную почту и аватар пользователя, а также позволяет задать настройки интеграции с Discord (адрес вебхука, имя отправителя, цвет сообщений). Дополнительные настройки пользователя хранятся в отдельном документе Firestore, связанном с идентификатором пользователя, и загружаются при открытии профиля. Предусмотрена проверка корректности настроек интеграции: пользователь может отправить тестовое сообщение и убедиться, что вебхук задан верно, прежде чем использовать автоматическую отправку результатов бросков. Эти настройки используются модулем бросков. Важно отметить, что в работе не приводятся реальные секретные значения: конфигурация подключения к Firebase рассматривается как техническая настройка, а адрес вебхука Discord — как пользовательское значение, вводимое на странице профиля и используемое только для отправки результатов бросков. Экран входа и профиля показан на рисунке 3.6.

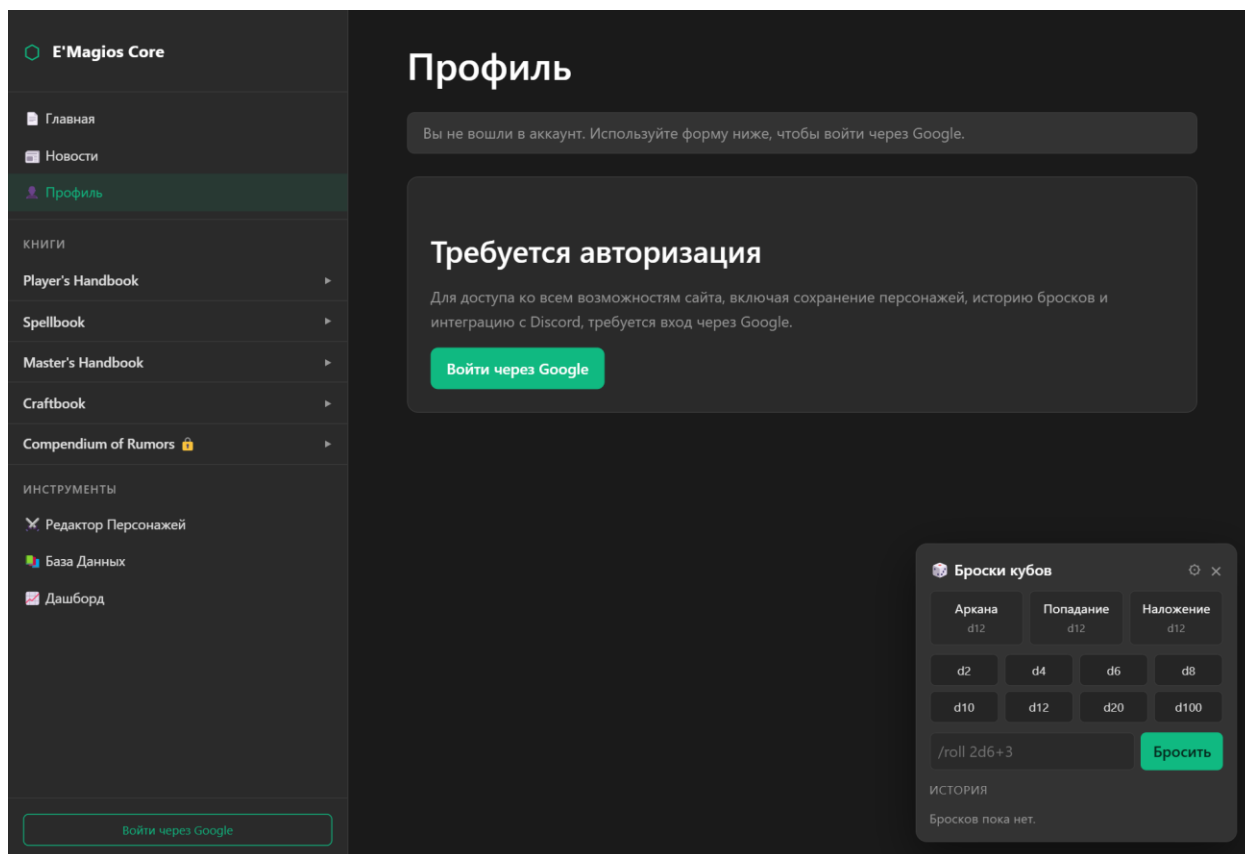


Рисунок 3.6 — Экран профиля пользователя с настройками интеграции

3.9 Дашборд данных

Дашборд (`pages/dashboard`, `features/dashboard`) предназначен для анализа данных платформы и доступен пользователям с ролью редактора или администратора. Числовая логика вынесена в отдельный модуль `diceAnalytics` и покрыта тестами. Дашборд состоит из трёх блоков.

Блок контента отображает сводные показатели справочника: общее число заклинаний и школ, долю заклинаний с концентрацией и подзаклинаний, количество неполных объектов и школ без заклинаний, а также распределение заклинаний по школам в виде гистограммы. Эти показатели помогают автору контролировать полноту и согласованность материалов системы.

Блок аналитики бросков агрегирует события бросков кубов: рассчитываются метрики по каждому типу куба — среднее выпавшее значение, теоретическое математическое ожидание, отклонение фактического среднего от теоретического и доля критических результатов; приводится

статистика по типам бросков и по пользователям. Сопоставление фактического среднего с теоретическим позволяет оценить, соответствует ли поведение генератора случайных значений ожидаемому распределению.

Блок качества данных выводит результаты автоматической проверки контента по коллекциям — количество ошибок, предупреждений и информационных замечаний, а также наиболее частые правила проверки. Этот блок служит инструментом контроля для автора и редактора: он позволяет своевременно выявлять неполные или некорректные записи и поддерживать справочник в согласованном состоянии. Источниками данных служат отчёт о данных, формируемый контентным конвейером, и события бросков. Числовая логика всех блоков вынесена в отдельный модуль и покрыта тестами, что гарантирует совпадение отображаемых метрик с эталонными расчётами на тех же данных. Экран дашборда показан на рисунке 3.7.

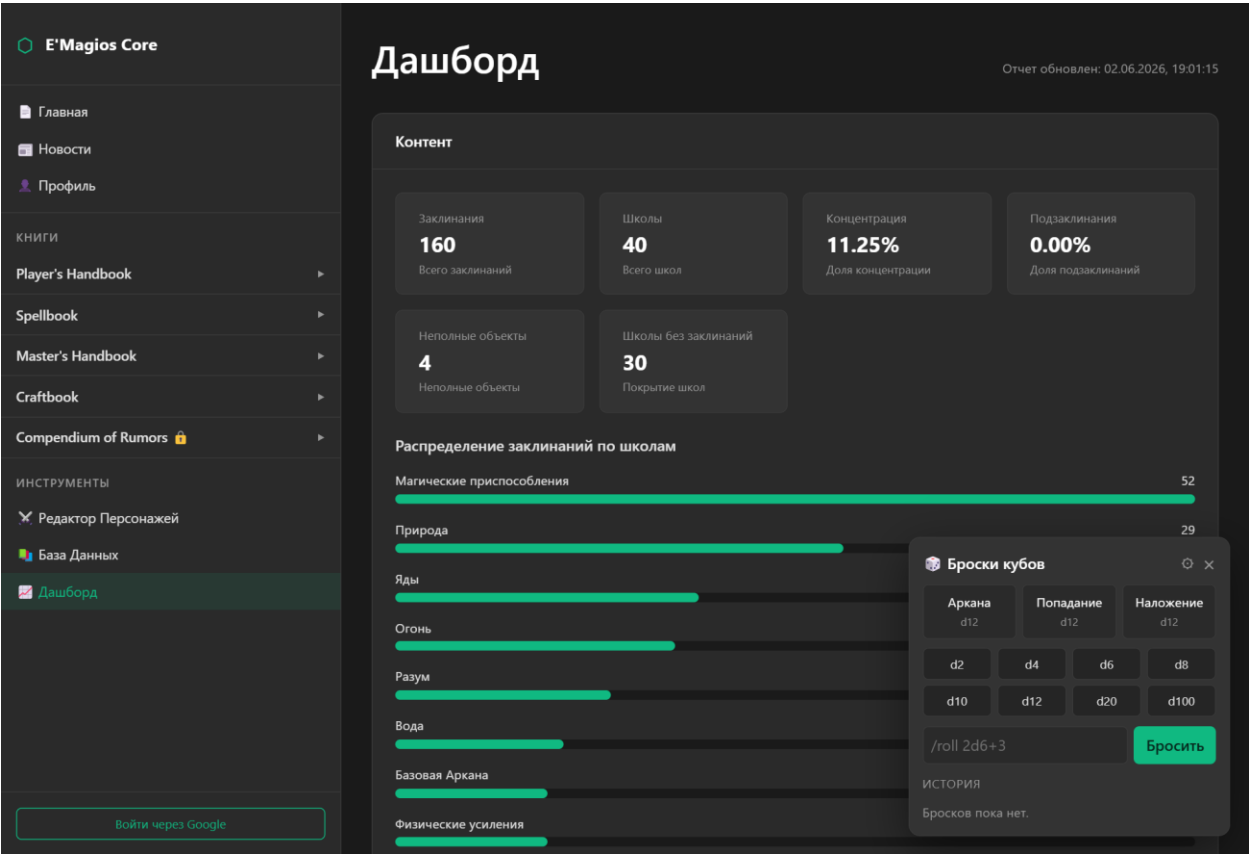


Рисунок 3.7 — Аналитический дашборд — блоки контента, бросков и качества данных

3.10 Главная страница и новости

Главная страница (HomePage) собрана из примитивов дизайн-системы и содержит секции-ссылки на основные разделы (база данных, редактор, новости, профиль) и карточки книг правил со ссылкой на первую главу и пометкой закрытых книг. Главная служит точкой входа, дающей пользователю целостное представление о возможностях платформы и обеспечивающей быстрый переход в любой раздел.

Новости (NewsPage) загружаются из контентного контура через репозиторий и отображаются с единым рендером текста, поддерживающим кликабельные ссылки на объекты. Раздел новостей информирует пользователей об обновлениях системы — добавлении заклинаний, изменении правил, новых возможностях платформы; ссылки на упомянутые игровые объекты открывают их карточки через общий оверлей, что связывает новости с базой знаний. Содержимое новостей может формироваться на этапе сборки на основании актуальных данных компендиума, благодаря чему перечни упомянутых объектов и сводная статистика остаются согласованными с базой и не требуют ручного обновления. Применение общего рендера текста к новостям, описаниям сущностей и тексту книг обеспечивает единообразное поведение ссылок и формул во всех текстовых блоках приложения. Главная страница показана на рисунке 3.8.

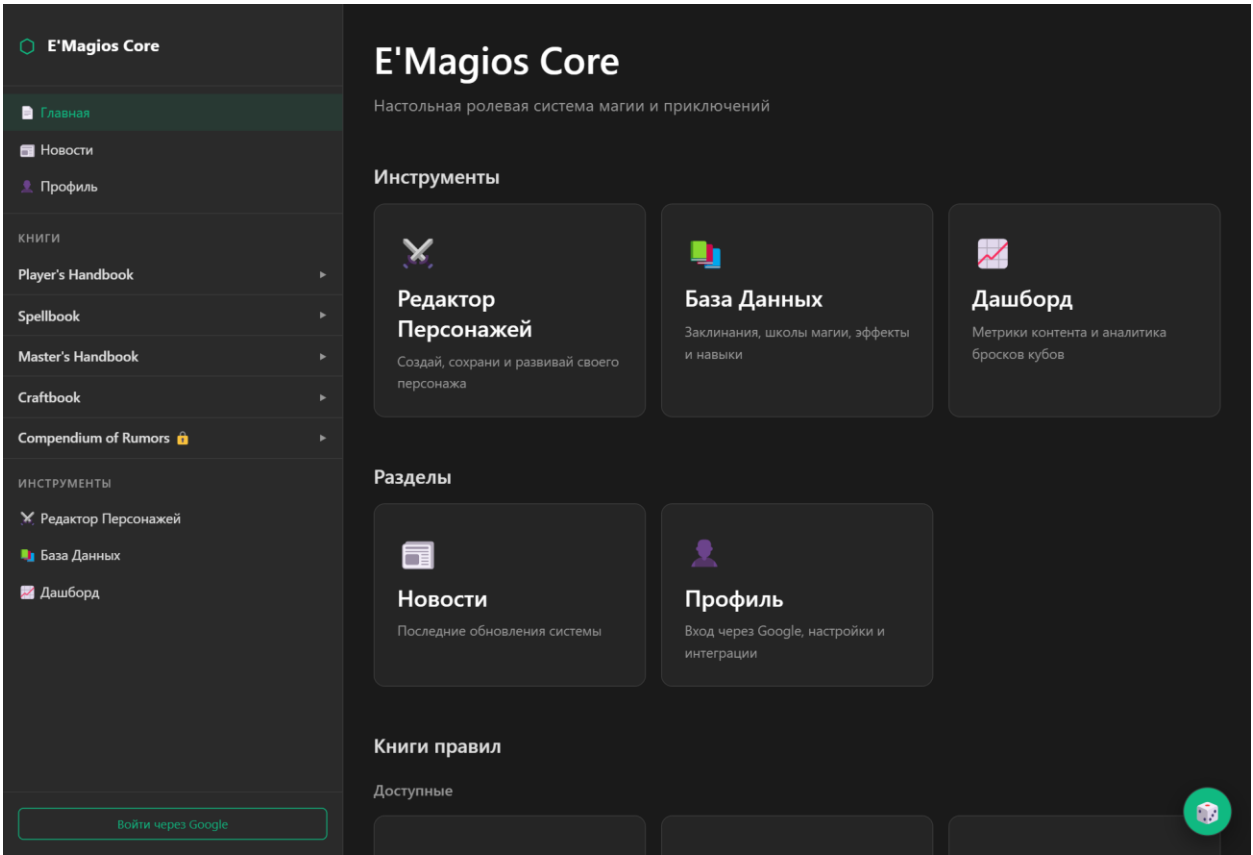


Рисунок 3.8 — Главная страница — разделы и карточки книг правил

3.11 Тестирование

Контроль корректности обеспечивается модульными тестами (фреймворк Vitest) [20] и сквозной проверкой качества. Стратегия тестирования сознательно сфокусирована на чистой бизнес-логике, а не на отображении: тестируется то, что вычисляет результат и подвержено ошибкам при переносе, тогда как визуальное оформление проверяется при разработке. Такой выбор обеспечивает высокую отдачу от тестов при умеренных затратах на их сопровождение. Тестами покрыта ключевая бизнес-логика, перенос которой наиболее подвержен ошибкам: расчёт характеристик персонажа, парсер выражений бросков, метрики дашборда, фильтрация и сортировка компендиума, разрешение кросс-ссылок и преобразование ссылок книг. Сводка покрытия приведена в таблице 5.

Таблица 5 — Покрытие модульными тестами

Модуль	Что проверяется	Тестовый файл
characterCalculations	расчёт характеристик 1–20	characterCalculations.test.ts

rollExpression	парсер и вычисление бросков, криты	rollExpression.test.ts
diceAnalytics	метрики бросков и контента	diceAnalytics.test.ts
useCompendiumSort	сортировка сущностей	useCompendiumSort.test.ts
useCompendiumIndex	резолв кросс-ссылок	useCompendiumIndex.test.ts
books	преобразование ссылок книг	books.test.ts
Репозитории	выборка и маппинг данных	CompendiumRepository.test.ts

Выбор модульного тестирования именно для этой логики обусловлен характером рисков переноса. Наибольшую опасность представляет незаметное искажение игровых правил: ошибка в таблице характеристик или в порядке применения бонусов не приводит к падению приложения, но делает результаты некорректными. Такие дефекты трудно обнаружить визуально, поэтому формулы зафиксированы тестами, сверяющими расчёт с эталонными значениями правил системы для всех уровней. Аналогично парсер выражений бросков проверяется на корректных и заведомо некорректных строках, включая граничные случаи — пустую команду, недопустимые грани кубов, два знака подряд. Внедрение генератора случайных чисел в роллер сделано специально для тестируемости: в тестах вместо случайного источника подаётся предопределённая последовательность, что позволяет проверить вычисление результата детерминированно.

Все тесты выполняются автоматически в составе команды `npm run check` и в конвейере непрерывной интеграции. Изменения, нарушающие тесты или проверку типов, не публикуются. Такой подход гарантирует, что перенос бизнес-логики из ранней версии выполнен без искажения правил системы, а дальнейшие доработки не нарушают уже проверенное поведение.

3.12 Развёртывание и непрерывная интеграция

Развёртывание платформы организовано как полностью автоматизированный процесс. Исходный код хранится в системе контроля

версий, разработка ведётся в отдельных ветках, а изменения попадают в основную ветку через запросы на слияние. Каждое изменение проходит конвейер непрерывной интеграции, который выполняет установку зависимостей, проверку качества кода и сборку приложения. Только при успешном прохождении всех проверок собранное приложение публикуется на статическом хостинге.

Такая организация даёт несколько практических преимуществ. Во-первых, исключается человеческий фактор при публикации: процесс воспроизводим и не зависит от ручных действий. Во-вторых, проверка качества перед публикацией не позволяет выложить в эксплуатацию код с ошибками типов, нарушениями стиля или непройденными тестами. В-третьих, поскольку приложение публикуется как набор статических файлов, оно не требует выделенного сервера приложения, отличается высокой доступностью и низкой стоимостью эксплуатации.

Поскольку приложение использует хеш-маршрутизацию, статический хостинг корректно обслуживает все маршруты без дополнительной серверной настройки: любой прямой переход и обновление страницы обрабатываются на стороне клиента. Контент книг правил и отчёт о данных размещаются как статические ресурсы и обновляются в составе того же конвейера, что обеспечивает согласованность кода и данных в каждой публикации. Эксплуатационное наблюдение за приложением обеспечивается встроенной телеметрией, события которой доступны во внутреннем служебном разделе и позволяют отслеживать переходы между разделами и характеристики загрузки данных.

3.13 Выводы по главе 3

В практической части выполнена программная реализация платформы. Организована структура проекта по слоям FSD со сборкой Vite, единым контролем качества и автоматической публикацией на GitHub Pages. Реализован слой shared с дизайн-системой, репозиториями, кэшированием и телеметрией. Реализованы функциональные срезы: база знаний с

четырнадцатью категориями, поиском и кросс-ссылками; книги правил с преобразованием внутренних ссылок; редактор персонажей с проверенными расчётами и облачным сохранением; глобальный виджет бросков с интеграцией Discord; сквозные оверлей деталей и единый рендер текста; авторизация с разграничением ролей и профиль; аналитический дашборд; главная страница и новости. Ключевая бизнес-логика покрыта модульными тестами. Полученный программный продукт соответствует поставленным цели и задачам. Демонстрационные материалы (слайды презентации) приведены в приложении Б, а полные тексты ключевых листингов и правил безопасности — в приложении В.

Заключение

В ходе выполнения выпускной квалификационной работы разработана веб-платформа E'Magios Core — многофункциональное типобезопасное одностраничное веб-приложение для цифрового сопровождения авторской настольной ролевой системы. Ключевой особенностью работы является перевод проекта с архитектуры набора статических страниц и разрозненных скриптов на строгую компонентную архитектуру с разделением на слои, статической типизацией и автоматизированным контролем качества. Это качественно повысило надёжность, расширяемость и сопровождаемость продукта, сохранив и развив его пользовательскую функциональность. В процессе работы успешно пройдены этапы анализа, проектирования, разработки и тестирования, что позволило достичь соответствия поставленным цели и задачам. По результатам работы получены следующие выводы:

- проанализирована предметная область цифровых инструментов для настольных ролевых систем, определена целевая аудитория и проведён сравнительный анализ аналогичных решений, выявивший нишу платформы, объединяющей справочный и интерактивный функционал;
- обоснован выбор технологического стека (React, TypeScript, Vite, Firebase, IndexedDB) и архитектурной методологии Feature-Sliced Design;
- спроектирована слоистая архитектура приложения, модель данных с четырнадцатью категориями справочных сущностей, статусная модель жизненного цикла контента и разграничение доступа на основе ролей;
- спроектирован конвейер подготовки контента из авторской среды Obsidian, обеспечивающий единый источник правды;
- разработана разделяемая дизайн-система и реализованы функциональные модули: база знаний, книги правил, редактор персонажей, глобальный виджет бросков кубов, авторизация и профиль, аналитический дашборд, главная страница и новости;

- реализованы облачное хранение данных и разграничение доступа на основе ролей через пользовательские утверждения токена и правила безопасности Firestore;
- ключевая бизнес-логика покрыта модульными тестами, организован непрерывный контроль качества кода;
- проведено тестирование, подтвердившее корректность работы основных модулей и соответствие требованиям.

Отдельно следует отметить методическую ценность проделанной работы. Переход от ранней версии на современную архитектуру выполнен не как переписывание «с нуля», а как управляемая миграция с сохранением и проверкой бизнес-правил: формулы и логика переносились по одной функции, сверялись с исходным поведением и закреплялись тестами. Такой подход к миграции, при котором источником истины служит работающая прежняя версия, а каждый перенесённый фрагмент подтверждается тестом, позволил избежать искажения правил системы и может быть применён при модернизации других программных продуктов.

Практическая значимость работы состоит в том, что создан действующий программный продукт, готовый к использованию игроками и ведущими настольной ролевой системы E'Magios Core и обеспечивающий реальные потребности игрового процесса — доступ к правилам, ведение персонажей и выполнение бросков. Одновременно работа имеет и более широкое значение как пример применения современной фронтенд-архитектуры и методологии Feature-Sliced Design к предметной области с богатой и развивающейся доменной моделью, а также как образец управляемой миграции существующего продукта на новый технологический стек с сохранением и проверкой бизнес-правил.

Реализованная архитектура, основанная на строгом слоистом разделении кода, статической типизации, изолированном слое доступа к данным и покрытой тестами бизнес-логике, демонстрирует гибкость и масштабируемость, что упрощает дальнейшее развитие проекта.

Перспективы развития платформы связаны с несколькими направлениями. Первое — расширение справочного контента и создание средств его редактирования непосредственно в приложении, что позволит авторам и редакторам работать с материалами через веб-интерфейс, опираясь на уже спроектированную статусную модель и систему ролей. Второе — мобильная адаптация интерфейса для использования платформы за игровым столом с планшетов и телефонов. Третье — углубление аналитического дашборда и переход на хранение событий бросков в облаке для сквозной аналитики между устройствами. Четвёртое — развитие средств сетевого взаимодействия игроков для совместного ведения игрового процесса в реальном времени. Заложенная слоистая архитектура и изоляция функций делают добавление каждого из этих направлений локальным изменением, не затрагивающим существующую функциональность. Полученные результаты создают технологическую основу для масштабирования цифровой экосистемы проекта E'Magios Core.

Библиографический список

- 1) React: The library for web and native user interfaces [Электронный ресурс]. — Официальная документация. — URL: <https://react.dev/> (дата обращения: 03.06.2026).
- 2) TypeScript: JavaScript with syntax for types [Электронный ресурс]. — Официальная документация. — URL: <https://www.typescriptlang.org/docs/> (дата обращения: 03.06.2026).
- 3) ECMAScript 2024 Language Specification [Электронный ресурс]. — URL: <https://tc39.es/ecma262/> (дата обращения: 03.06.2026).
- 4) Хавербеке М. Выразительный JavaScript. Современное веб-программирование / М. Хавербеке. — 3-е изд. — Санкт-Петербург : Питер, 2019. — 480 с.
- 5) Vite: Next Generation Frontend Tooling [Электронный ресурс]. — Официальная документация. — URL: <https://vitejs.dev/> (дата обращения: 03.06.2026).
- 6) React Router [Электронный ресурс]. — Официальная документация. — URL: <https://reactrouter.com/> (дата обращения: 03.06.2026).
- 7) Feature-Sliced Design: Architectural methodology for frontend projects [Электронный ресурс]. — Официальная документация. — URL: <https://feature-sliced.design/> (дата обращения: 03.06.2026).
- 8) Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. — Санкт-Петербург : Питер, 2018. — 352 с.
- 9) Firebase Authentication [Электронный ресурс]. — Документация Google Firebase. — URL: <https://firebase.google.com/docs/auth> (дата обращения: 03.06.2026).
- 10) Cloud Firestore [Электронный ресурс]. — Документация Google Firebase. — URL: <https://firebase.google.com/docs/firestore> (дата обращения: 03.06.2026).

- 11) Firebase Security Rules [Электронный ресурс]. — Документация Google Firebase. — URL: <https://firebase.google.com/docs/rules> (дата обращения: 03.06.2026).
- 12) IndexedDB API [Электронный ресурс]. — MDN Web Docs. — URL: https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API (дата обращения: 03.06.2026).
- 13) idb: IndexedDB with usability [Электронный ресурс]. — Репозиторий проекта. — URL: <https://github.com/jakearchibald/idb> (дата обращения: 03.06.2026).
- 14) MDN Web Docs [Электронный ресурс]. — Справочник по веб-технологиям. — URL: <https://developer.mozilla.org/> (дата обращения: 03.06.2026).
- 15) Obsidian: A knowledge base that works on local Markdown files [Электронный ресурс]. — Официальная документация. — URL: <https://help.obsidian.md/> (дата обращения: 03.06.2026).
- 16) GitHub Pages Documentation [Электронный ресурс]. — URL: <https://docs.github.com/en/pages> (дата обращения: 03.06.2026).
- 17) GitHub Actions Documentation [Электронный ресурс]. — URL: <https://docs.github.com/en/actions> (дата обращения: 03.06.2026).
- 18) Фримен Э. Head First. Паттерны проектирования / Э. Фримен, Э. Робсон. — Санкт-Петербург : Питер, 2020. — 656 с.
- 19) ESLint: Pluggable JavaScript linter [Электронный ресурс]. — Официальная документация. — URL: <https://eslint.org/docs/latest/> (дата обращения: 03.06.2026).
- 20) Vitest: A Vite-native testing framework [Электронный ресурс]. — Официальная документация. — URL: <https://vitest.dev/> (дата обращения: 03.06.2026).

Приложение А. Техническое задание

А.1 Наименование и область применения

Наименование разработки — веб-платформа для настольной ролевой системы «E'Magios Core». Область применения — цифровое сопровождение настольных ролевых игр: предоставление игрокам и ведущим структурированного справочника игровых материалов, электронных книг правил, редактора персонажей и инструмента бросков кубов с облачным хранением пользовательских данных.

А.2 Основания для разработки

Основанием для разработки является задание на выпускную квалификационную работу, утверждённое кафедрой «Информатика и информационные технологии» Московского политехнического университета. Тема ВКР: «Разработка сайта для настольно-ролевой игры».

А.3 Назначение разработки

Назначение системы — объединить в единой типобезопасной веб-среде справочный контент авторской ролевой системы и персональные интерактивные инструменты игрока. Цель создания системы — обеспечить удобный доступ к игровым материалам, автоматизировать расчёт характеристик персонажа, предоставить прозрачный модуль бросков кубов и разграничить доступ к закрытым материалам на основе ролей.

А.4 Требования к программе

А.4.1 Требования к функциональным характеристикам

Система должна обеспечивать выполнение следующих функций:

- ведение и отображение структурированной базы игровых сущностей (четырнадцать категорий) с поиском, фильтрацией и сортировкой;
- отображение электронных книг правил с навигацией, якорями и взаимными ссылками;
- создание, редактирование, импорт, экспорт и облачное хранение персонажей;

- автоматический расчёт производных характеристик персонажа по уровню (1–20);
- выполнение бросков кубов по выражениям вида $2d4+3d6-1$ с учётом бонусов персонажа;
- отправку результатов бросков во внешний сервис Discord по технологии вебхуков;
- авторизацию пользователей через аккаунт Google и разграничение доступа по ролям (автор, редактор, администратор);
- аналитический дашборд с метриками контента, бросков и качества данных.

А.4.2 Требования к входным и выходным данным

Входными данными являются справочные материалы ролевой системы, подготавливаемые в авторской среде Obsidian (формат Markdown) и преобразуемые в структурированные документы облачной базы, а также пользовательский ввод (данные персонажей, выражения бросков, настройки интеграции). Выходными данными являются отображаемые справочные карточки, рассчитанные характеристики персонажа, результаты бросков и агрегированные показатели дашборда.

А.4.3 Требования к надёжности и безопасности

Контроль доступа к данным должен обеспечиваться декларативными правилами безопасности на стороне сервера, проверяемыми независимо от клиента. Ключевая бизнес-логика (расчёт характеристик, парсер бросков, метрики аналитики) должна быть покрыта модульными тестами. Отображение внешнего HTML-содержимого должно выполняться с санитайзингом.

А.4.4 Требования к составу и параметрам технических средств

Клиентская часть должна работать в современном веб-браузере (Google Chrome, Mozilla Firefox, Microsoft Edge актуальных версий) на персональном компьютере или мобильном устройстве. Серверная часть не требует выделенного сервера приложения: приложение публикуется как набор

статических файлов, а функции хранения и аутентификации обеспечиваются облачными сервисами Firebase.

А.4.5 Требования к составу программного обеспечения

Стек разработки: React 18, TypeScript, Vite, React Router; облачные сервисы Firebase Authentication и Cloud Firestore; клиентское кэширование IndexedDB; модульное тестирование Vitest; контроль версий Git и публикация через GitHub Pages с конвейером непрерывной интеграции GitHub Actions.

А.5 Стадии и этапы разработки

Таблица А.1 — Стадии и этапы разработки

Этап	Содержание этапа
Исследование предметной области	Анализ цифровых инструментов НРИ, аналогов, целевой аудитории
Разработка технического задания	Формулирование функциональных и нефункциональных требований
Эскизное и техническое проектирование	Проектирование архитектуры FSD, модели данных, интерфейсов
Программная реализация	Реализация дизайн-системы и функциональных модулей платформы
Тестирование	Модульное тестирование бизнес-логики, контроль качества в CI
Развёртывание	Публикация на GitHub Pages, настройка облачных сервисов

А.6 Техническая документация

По окончании работы предоставляются: пояснительная записка к ВКР, исходный код проекта в системе контроля версий, набор автоматизированных тестов и презентация результатов.

Приложение Б. Презентация

Федеральное государственное автономное
образовательное учреждение высшего
образования «Московский политехнический
университет»

Разработка сайта для настольно-ролевой игры

Автор: Гайдарь Максим Денисович
Студент группы 221-3711
Дипломный руководитель: Кандрашина М. А.
Старший преподаватель



Слайд 1

Разработка сайта для настольно-ролевой игры

Актуальность и новизна

Объект: процесс цифровой поддержки настольной ролевой системы E'Magios Core.

Предмет: веб-платформа для хранения, поиска и использования правил, справочников и пользовательских персонажей.

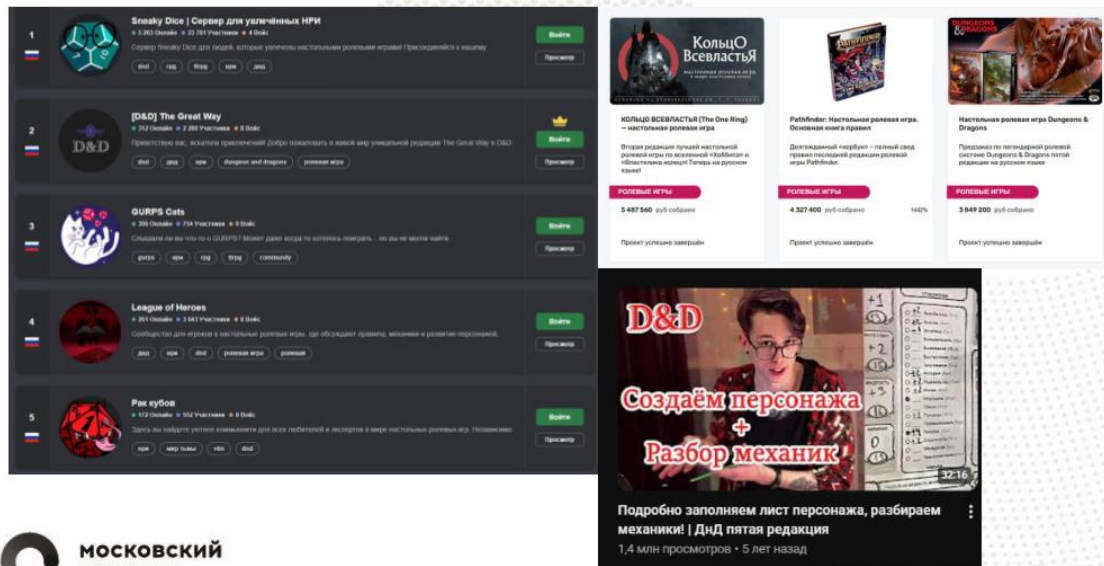
Актуальность: настольная ролевая система содержит большой объём правил, заклинаний и данных персонажей. Веб-платформа упрощает поиск информации, ведение персонажей и обеспечивает облачный доступ с разных устройств.



2

Разработка сайта для настольно-ролевой игры

Актуальность и новизна



Разработка сайта для настольно-ролевой игры

Цель и задачи

Цель работы – разработка многофункционального веб-сайта для поддержки настольной ролевой системы E'Magios Core, обеспечивающего удобный доступ к игровому контенту и предоставляющего пользователям интерактивные инструменты для подготовки и сопровождения игрового процесса.

Задачи:

1. Исследование предметной области
2. Анализ аналогичных продуктов
3. Создание парсинга для переноса переноса контента из Obsidian
4. Проектирование структуры веб-приложения
5. Разработка веб-сайта
6. Тестирование

Разработка сайта для настольно-ролевой игры

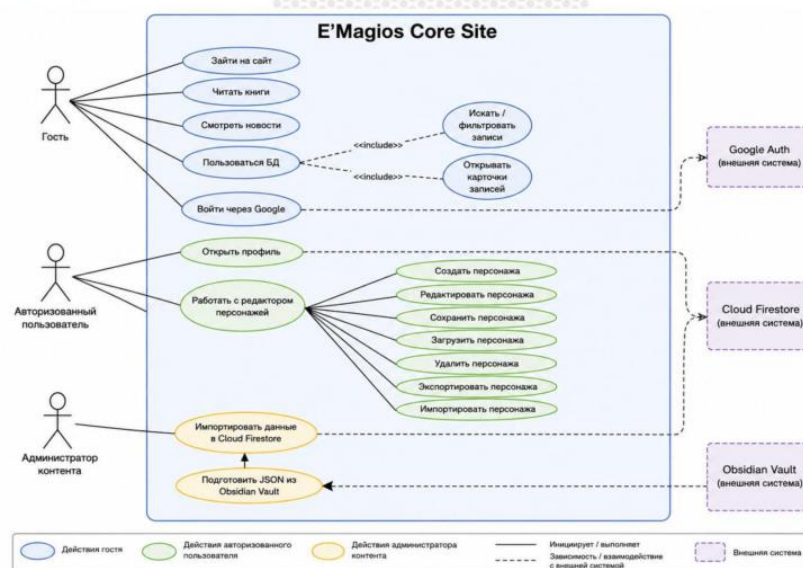
Анализ конкурентов

Критерий	dnd.su	ttg.club	Long Story Short	E'Magios Core (наш сайт)
Справочная база (D&D контент)	✓	✓	✗	✓
Поиск и фильтрация	✓	✓	✗	✓
Игровые инструменты (листы, броски и др.)	✗	✓	✓	✓
Пользовательский контент и настройки	✗	✗	✓	✓
Интеграция с внешними сервисами	✗	✗	✓	✓
Основной фокус	Справочник по D&D	Справочник + инструменты	Игровые инструменты	Справочник + инструменты в одной платформе

Слайд 5

Разработка сайта для настольно-ролевой игры

Функциональные возможности



Слайд 6

Разработка сайта для настольно-ролевой игры

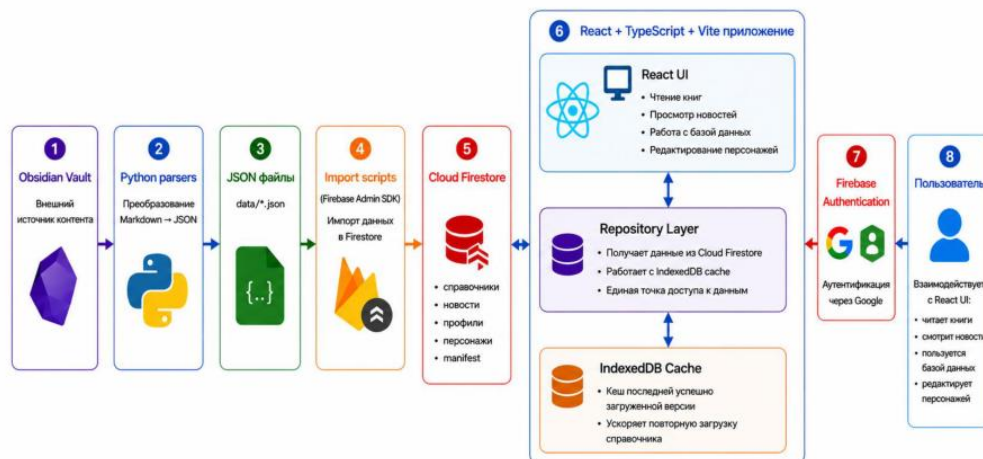
Инструментальные средства



Слайд 7

Разработка сайта для настольно-ролевой игры

Архитектура



Слайд 8

Разработка сайта для настольно-ролевой игры

БД

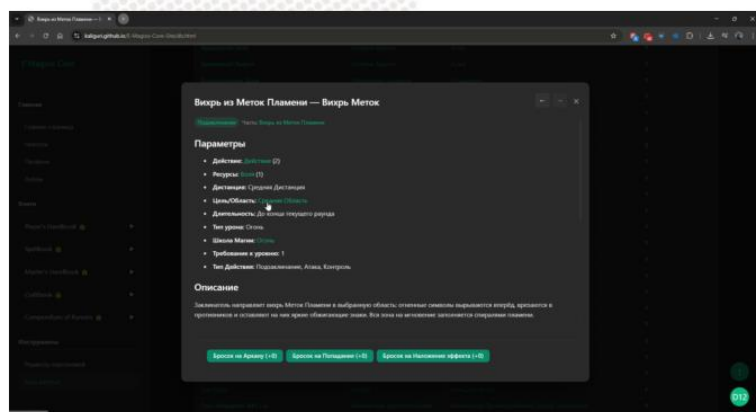
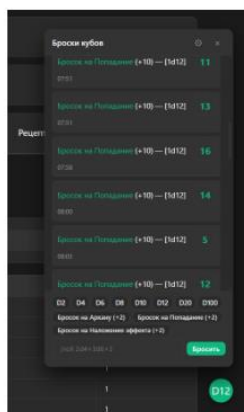


Слайд 9

Разработка сайта для настольно-ролевой игры

Основные элементы веб-приложения

Проект представляет собой многофункциональную веб-платформу, включающую базу данных игровых материалов, редактор персонажей, систему бросков кубов, интеграцию с Discord.

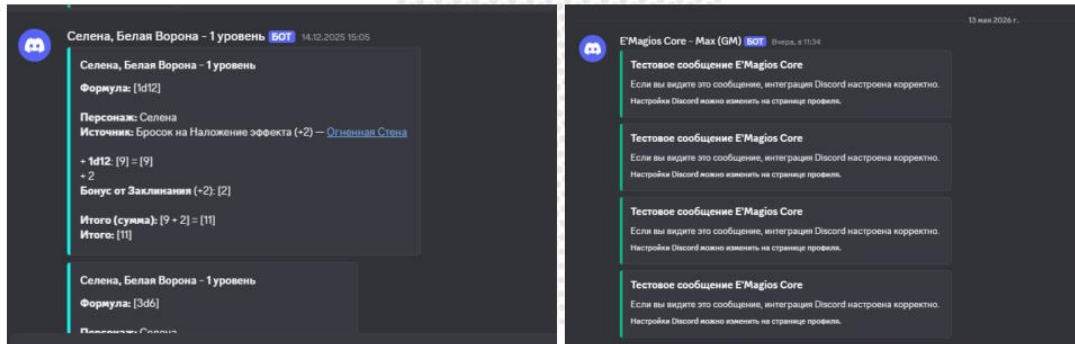


Слайд 10

Разработка сайта для настольно-ролевой игры

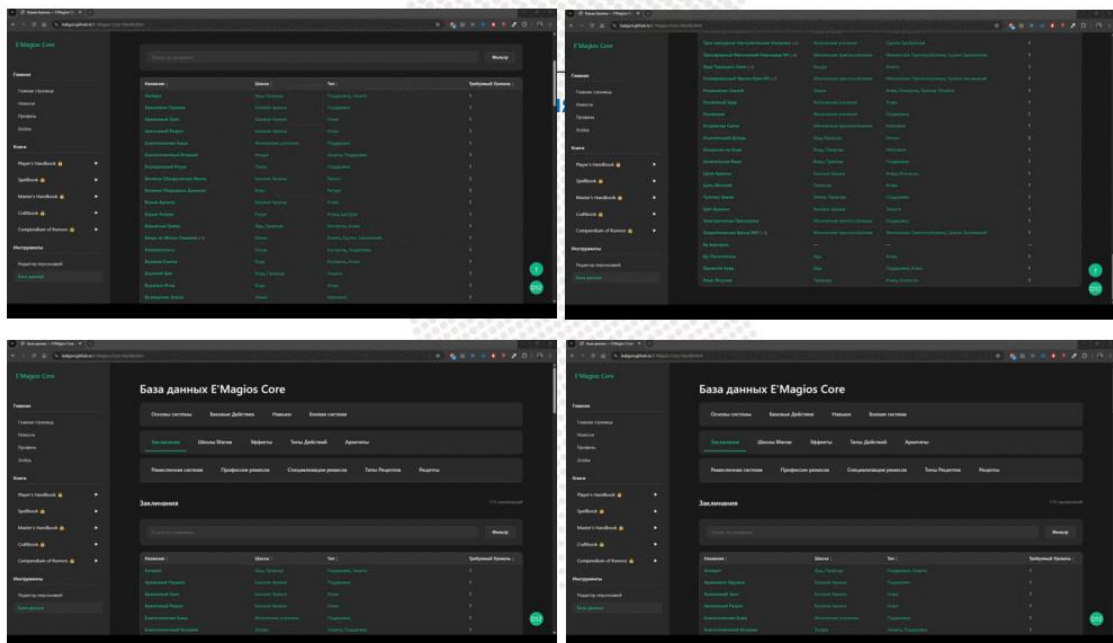
Основные элементы веб-приложения

Проект представляет собой многофункциональную веб-платформу, включающую базу данных игровых материалов, редактор персонажей, систему бросков кубов, интеграцию с Discord.



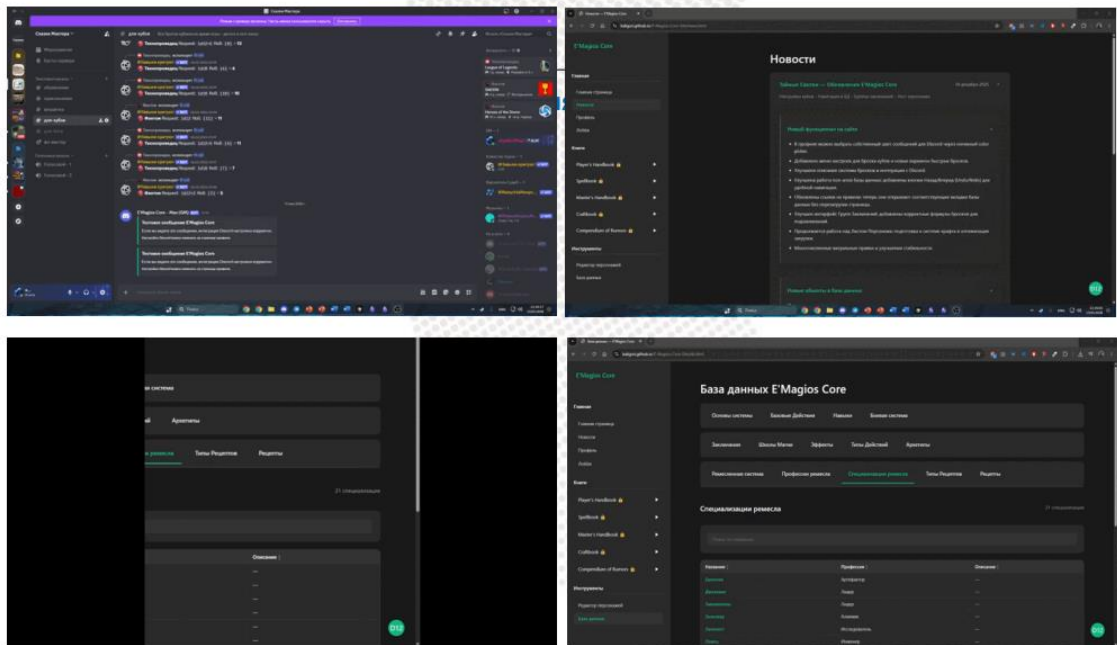
11

Слайд 11



12

Слайд 12



Слайд 13

Разработка сайта для настольно-ролевой игры

Тестирование и результаты

Оптимизация:

1. Проведение Usability Testing;
2. Единый Repository Layer для доступа к Firestore;
3. Удаление JSON fallback из runtime;
4. Кеширование справочника через IndexedDB;

Слайд 14

Разработка сайта для настольно-ролевой игры

Заключение

В ходе выполнения выпускной квалификационной работы был разработан веб-сайт E'Magios Core. В процессе создания программного продукта были успешно выполнены этапы анализа, проектирования, разработки и тестирования, что позволило достичь соответствия поставленным целям и задачам.

Перспективы: Планируется добавление Редактора Заклинаний, создание Content Editor, интеграция с Telegram, автоматические тесты.



15

Слайд 15

Спасибо за внимание!

«Разработка сайта для настольно-ролевой игры»

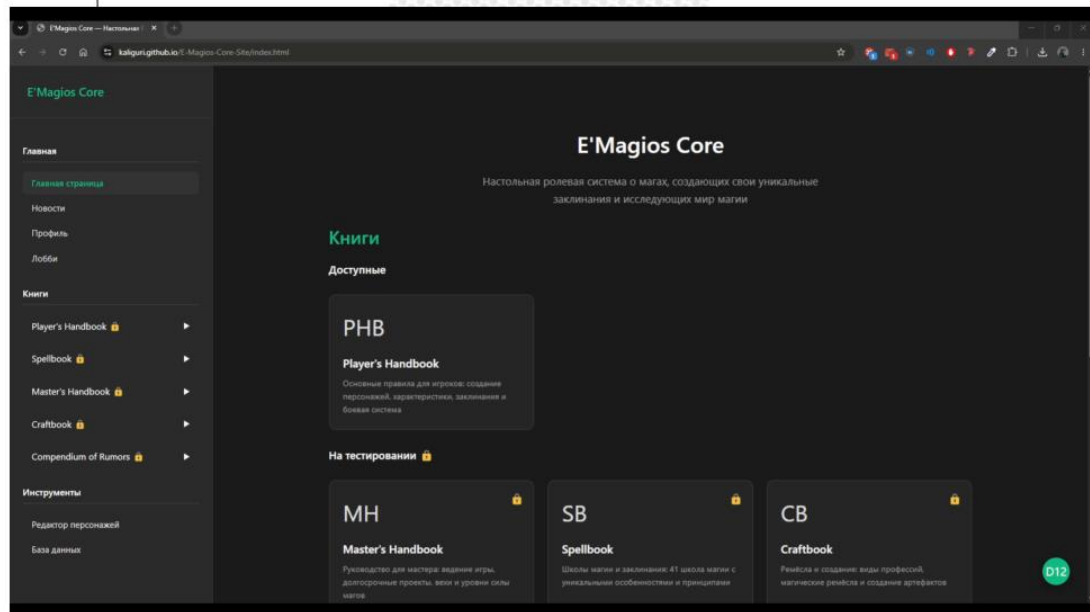
Автор: Гайдарь Максим Денисович
Студент группы 221-3711

Дипломный руководитель: Кандрашина М. А.
Старший преподаватель



Слайд 16

Разработка сайта для настольно-ролевой игры



Приложение В. Листинги программного кода

В.1 Правила безопасности Cloud Firestore

Листинг В.1 — Правила доступа к данным по ролям и статусам

(firestore.rules, сокращённо)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function userRole() {
      return request.auth != null && request.auth.token.role is string
        ? request.auth.token.role : '';
    }
    function isAuthor() { return userRole() == 'author'; }
    function isEditor() { return userRole() == 'editor'; }
    function isAdmin() { return userRole() == 'admin'; }
    function isOwner(uid) {
      return request.auth != null && request.auth.uid == uid;
    }
    function isValidStatus(status) {
      return status in ['draft', 'review', 'published', 'archived'];
    }
    function canCreateContent() {
      return request.auth != null && (isAuthor() || isEditor() || isAdmin())
        && isValidStatus(request.resource.data.status)
        && request.resource.data.status == 'draft';
    }
    function canUpdateContent() {
      return request.auth != null && (isAuthor() || isEditor() || isAdmin())
        && isValidStatus(resource.data.status)
        && isValidStatus(request.resource.data.status) && (
          (isAuthor() && resource.data.status == 'draft'
            && request.resource.data.status in ['draft', 'review']) ||
          (isEditor() && (
            resource.data.status == 'review'
            && request.resource.data.status in ['draft', 'published']) ||
            (resource.data.status == 'published'
            && request.resource.data.status == 'archived')
          )) || isAdmin()
        );
    }
    // Справочные коллекции: публичное чтение опубликованного,
    // запись по ролям (импорт идёт через Admin SDK в обход правил)
    match /spells/{docId} {
      allow read: if resource.data.status == 'published';
      allow create: if canCreateContent();
      allow update: if canUpdateContent();
      allow delete: if isAdmin();
    }
    // ... остальные 13 категорий компендиума и news – аналогично ...

    // Данные пользователя – доступ только владельцу
    match /users/{userId} {
      allow read, write: if isOwner(userId);
      match /characters/{characterId} {
        allow read, write: if isOwner(userId);
      }
    }
  }
}
```

```

    // Запрет всего остального
    match /{document=**} { allow read, write: if false; }
  }
}

```

В.2 Парсер и вычислитель выражений броска

Листинг В.2 — Чистый парсер и роллер с инъекцией ГСЧ (rollExpression.ts, фрагмент)

```

const ALLOWED_SIDES = [2, 4, 6, 8, 10, 12, 20, 100];

export function parseRollExpression(raw: string): ParsedRoll {
  const trimmed = raw.trim();
  const withoutCommand = trimmed.toLowerCase().startsWith('/roll')
    ? trimmed.slice(5).trim() : trimmed;
  if (!withoutCommand) throw new Error('Пустая команда броска.');
```

const normalized = withoutCommand.replace(/\\s+/g, '');

if (!/^([0-9dD+\\-*/]+)\$/i.test(normalized)) {

throw new Error('Допускаются только цифры, d, +, -, * и /.');

}

// ... разбиение на сегменты по знакам + и - ...

const m = text.match(/^(\\d*)(dD)(\\d+)(?:([*/])(\\d+))?\$)/);

const count = m[1] ? parseInt(m[1], 10) : 1;

const sides = parseInt(m[2], 10);

if (count <= 0 || count > 100) {

throw new Error('Количество кубов должно быть от 1 до 100.');

}

if (!ALLOWED_SIDES.includes(sides)) {

throw new Error('Разрешены только D2, D4, D6, D8, D10, D12, D20 и D100.');

}

return { expression: normalized, segments: parsedSegments };

}

// Роллер: ГСЧ передаётся параметром – для детерминированных тестов

export function rollExpression(parsed: ParsedRoll,

rng: () => number = Math.random): RollResult {

let total = 0;

parsed.segments.forEach((seg) => {

if (seg.kind === 'dice') {

let baseSum = 0;

for (let i = 0; i < seg.count; i += 1) {

baseSum += 1 + Math.floor(rng() * seg.sides);

}

let segmentTotal = baseSum;

if (seg.scaleOp && seg.scale) {

segmentTotal = seg.scaleOp === '*'

? baseSum * seg.scale : baseSum / seg.scale;

}

total += seg.sign * segmentTotal;

} else {

total += seg.sign * seg.value;

}

});

return { expression: parsed.expression, total, parts, createdAt: Date.now() };

}

// Натуральная «12» на одиночном d12 – критический успех (кроме бросков наложения)

export function isCriticalRoll(result: RollResult, isApply = false): boolean {

if (isApply) return false;

```

    return result.parts.some((part) => part.kind === 'dice'
      && part.sides === 12 && part.count === 1 && part.rolls.includes(12));
  }
}

```

В.3 Расчёт производных характеристик персонажа

Листинг В.3 — Табличный расчёт характеристик по уровню

(characterCalculations.ts, фрагмент)

```

// Таблица характеристик по уровню (1-20):
// arcana, health, will, speed, initiative, hitBonus,
// effectBonus, evasion, fortitude, actions, reactions
const STAT_TABLE: Record<number, Omit<CharacterStats, 'level'>> = {
  1: { arcana: 1, health: 6, will: 4, speed: 6, initiative: 1,
    hitBonus: 1, effectBonus: 1, evasion: 10, fortitude: 10, actions: 2,
    reactions: 1 },
  // ... уровни 2-19 ...
  20: { arcana: 10, health: 25, will: 14, speed: 10, initiative: 10,
    hitBonus: 11, effectBonus: 11, evasion: 25, fortitude: 25, actions: 5,
    reactions: 3 },
};

export function calculateStats(level: number): CharacterStats {
  const clamped = Math.max(1, Math.min(20, level));
  const base = STAT_TABLE[clamped] ?? STAT_TABLE[1];
  return { level: clamped, ...base };
}

```